

**BIRD SPECIES IDENTIFICATION USING AUDIO PROCESSING  
AND ALEXNET NEURAL NETWORK**

*Submitted for partial fulfillment of the requirements  
for the award of*

**BACHELOR OF TECHNOLOGY  
in  
COMPUTER SCIENCE & ENGINEERING**

by

**SANGAM SAI SRI VINAY REDDY - 19BQ1A05J7**

**THOTA CHANDRIKA - 19BQ1A05M9**

**SOMAROUTHU KAARTHIKEYA - 19BQ1A05L5**

**SHAIK SIDDIK - 19BQ1A05L0**

**TORLIKONDA SATYA AKHIL - 19BQ1A05P8**

Under the guidance of

**Dr. G. SANJAY GANDHI, M.Tech, Ph.D.**

**Professor**



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

(B.Tech Program is Accredited by NBA)

**VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY**

Permanently Affiliated to JNTU Kakinada, Approved by AICTE

Accredited by NAAC with 'A' Grade, ISO 9001:2008 Certified

NAMBUR(V), PEDAKAKANI(M), GUNTUR-522 508

Tel no: 0863-2118036, url: [www.vvitguntur.com](http://www.vvitguntur.com)

April 2023

## DECLARATION

We, Mr. SANGAM SAI SRI VINAY REDDY, Ms. THOTA CHANDRIKA, Mr. SOMAROUTHU KAARTHIKEYA PAVANA KUMAAR, Mr. SHAIK SIDDIK, Mr. TORLIKONDA SATYA AKHIL hereby declare that the Project Report entitled “**BIRD SPECIES IDENTIFICATION USING AUDIO PROCESSING AND ALEXNET NEURAL NETWORK**” done by us under the guidance of Dr. G. SANJAY GANDHI, Professor, CSE at Vasireddy Venkatadri Institute of Technology is submitted for partial fulfillment of the requirements for the award of Bachelor of Technology in Computer Science & Engineering. The results embodied in this report have not been submitted to any other University for the award of any degree.

DATE :

PLACE :

SIGNATURE OF THE CANDIDATE(S)

S. SAI SRI VINAY REDDY - 19BQ1A05J7

T. CHANDRIKA - 19BQ1A05M9

S. KAARTHIKEYA PAVANA KUMAAR - 19BQ1A05L5

Sk. SIDDIK - 19BQ1A05L0

T. SATYA AKHIL - 19BQ1QA05P8

## ACKNOWLEDGEMENT

We take this opportunity to express our deepest gratitude and appreciation to all those people who made this project work easier with words of encouragement, motivation, discipline, and faith by offering different places to look to expand our ideas and helped us towards the successful completion of this project work.

First and foremost, We express our deep gratitude to **Sri Vasireddy Vidya Sagar**, Chairman, Vasireddy Venkatadri Institute of Technology for providing necessary facilities throughout the B.Tech programme.

We express our sincere thanks to **Dr. Y. Mallikarjuna Reddy**, Principal, Vasireddy Venkatadri Institute of Technology for his constant support and cooperation throughout the B.Tech programme.

We express our sincere gratitude to **Dr. V. Ramachandran**, Professor & HOD, Computer Science & Engineering, Vasireddy Venkatadri Institute of Technology for his constant encouragement, motivation and faith by offering different places to look to expand my ideas.

We would like to express our sincere gratefulness to our guide **Dr. G. Sanjay Gandhi**, Professor, CSE, for his insightful advice, motivating suggestions, invaluable guidance, help and support in successful completion of this project.

We are sincerely thankful to our Project Coordinator **Dr. G. Sanjay Gandhi**, Professor, CSE for his valuable advices, motivating suggestions, moral support, help and coordination among us in successful completion of this project.

We would like to take this opportunity to express our thanks to the **teaching and non-teaching** staff in Department of Computer Science & Engineering, VVIT for their invaluable help and support.

**S. Sai Sri Vinay Reddy(19BQ1A05J7)**

**T. Chandrika(19BQ1A05M9)**

**S. Kaarthikeya Pavana Kumaar(19BQ1A05L5)**

**Sk. Siddik(19BQ1A05L0)**

**T. Satya Akhil(19BQ1A05P8)**



**VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY**

Permanently Affiliated to JNTU Kakinada, Approved by AICTE

Accredited by NAAC with 'A' Grade, ISO 9001:2008 Certified

Nambur, Pedakakani (M), Guntur (Dt) - 522508

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**

B.Tech Program is Accredited by NBA

## **CERTIFICATE**

This is to certify that this Project Report is the bonafide work of **SANGAM SAI SRI VINAY REDDY, THOTA CHANDRIKA, SOMAROUTHU KAARTHIKEYA PAVANA KUMAAR, SHAIK SIDDIK, TORLIKONDA SATYA AKHIL** bearing **Reg. No. 19BQ1A05J7, 19BQ1A05M9, 19BQ1A05L5, 19BQ1A05L0, 19BQ1A05P8** who had carried out the project entitled **“BIRD SPECIES IDENTIFICATION USING AUDIO PROCESSING AND ALEXNET NEURAL NETWORK”** under our supervision.

**Project Guide**

(Dr. G. Sanjay Gandhi, Professor)

**Head of the Department**

(Dr. V. Ramachandran, Professor)

---

**Submitted for Viva voce Examination held on \_\_\_\_\_**

**Internal Examiner**

**External Examiner**

# TABLE OF CONTENTS

| <b>Ch no.</b> | <b>Title</b>                           | <b>Page no.</b> |
|---------------|----------------------------------------|-----------------|
|               | Table of Contents                      | i               |
|               | List of Figures                        | iii             |
|               | List of Tables                         | iv              |
|               | Nomenclature                           | v               |
|               | Abstract                               | vi              |
| <b>1.</b>     | <b>INTRODUCTION</b>                    | <b>1</b>        |
| <b>2.</b>     | <b>AIM &amp; SCOPE</b>                 | <b>3</b>        |
|               | 2.1 Problem Description                | 3               |
|               | 2.2 Existing System                    | 3               |
|               | 2.2.1 Disadvantages of existing system | 3               |
|               | 2.3 Feasibility Study                  | 3               |
|               | 2.3.1 Economic Feasibility             | 4               |
|               | 2.3.2 Technical Feasibility            | 4               |
|               | 2.3.3 Social Feasibility               | 5               |
| <b>3.</b>     | <b>REVIEW OF LITERATURE</b>            | <b>6</b>        |
| <b>4.</b>     | <b>PROPOSED SOLUTION</b>               | <b>9</b>        |
|               | 4.1 Methodology                        | 9               |
| <b>5.</b>     | <b>SYSTEM REQUIREMENTS</b>             | <b>12</b>       |
|               | 5.1 Hardware Requirements              | 12              |
|               | 5.2 Software Requirements              | 13              |
| <b>6.</b>     | <b>SYSTEM DESIGN</b>                   | <b>15</b>       |
|               | 6.1 Usecase Diagram                    | 15              |
|               | 6.2 Class Diagram                      | 16              |
|               | 6.3 Sequence Diagram                   | 17              |
|               | 6.4 State chart Diagram                | 18              |
|               | 6.5 Activity Diagram                   | 19              |
|               | 6.6 Component Diagram                  | 20              |
|               | 6.7 Deployment Diagram                 | 21              |
| <b>7.</b>     | <b>IMPLEMENTATION</b>                  | <b>23</b>       |
|               | 7.1 Tools used                         | 23              |
|               | 7.2 Algorithms                         | 27              |
|               | 7.2.1 Modules                          | 27              |

|                                         |    |
|-----------------------------------------|----|
| 7.2.2 Algorithms                        | 28 |
| 7.2.3 Sample code                       | 30 |
| <b>8. RESULTS</b>                       | 36 |
| <b>9. TESTING</b>                       | 45 |
| 9.1 Phases of system testing            | 45 |
| 9.2 Test cases                          | 48 |
| <b>10. CONCLUSION</b>                   | 49 |
| <b>11. LIMITATIONS AND FUTURE SCOPE</b> | 50 |
| <b>12. REFERENCES</b>                   | 51 |
| <b>APPENDIX</b>                         | 53 |
| Conference Presentation Certificate     |    |
| Published Article on the Journal        |    |

## LIST OF FIGURES

| Sl. No. | Fig. No. | Fig. Name                               | Page No. |
|---------|----------|-----------------------------------------|----------|
| 1       | 1.1      | Process flow diagram                    | 1        |
| 2       | 4.1.1    | Silence removal                         | 9        |
| 3       | 4.1.2    | Flow chart                              | 11       |
| 2       | 6.1.1    | Use case diagram                        | 15       |
| 3       | 6.5.1    | Activity diagram                        | 19       |
| 4       | 6.2.1    | Class diagram                           | 16       |
| 5       | 6.3.1    | Sequence diagram                        | 17       |
| 6       | 6.4.1    | State chart diagram                     | 18       |
| 7       | 6.5.1    | Component diagram                       | 20       |
| 8       | 6.7.1    | Deployment diagram                      | 21       |
| 9       | 8.1      | Import packages                         | 36       |
| 10      | 8.2      | Noise cleaning                          | 37       |
| 11      | 8.3      | Identifying the species                 | 37       |
| 12      | 8.4      | Spectrogram of original signal          | 38       |
| 13      | 8.5      | Spectrogram after noise removal         | 38       |
| 14      | 8.6      | Extract spectrogram features            | 39       |
| 15      | 8.7      | Data visualization                      | 39       |
| 16      | 8.8      | Split data set for training and testing | 40       |
| 17      | 8.9      | Define performance metrics              | 40       |
| 18      | 8.10     | Build neural network using keras        | 41       |
| 19      | 8.11     | Confusion matrix                        | 42       |
| 20      | 8.12     | Performance metrics                     | 42       |
| 21      | 8.13     | Prediction function                     | 43       |
| 22      | 8.14     | Testing                                 | 43       |
| 23      | 9.1.1    | Static testing                          | 46       |
| 24      | 9.1.2    | Structural testing                      | 47       |
| 25      | 9.1.3    | Blackbox testing                        | 47       |

## LIST OF TABLES

| S. No. | Table No. | Table Description | Page No. |
|--------|-----------|-------------------|----------|
| 1      | 9.2.1     | Test cases        | 48       |



## NOMENCLATURE

| S. No. | Abbreviation | Explanation                                    |
|--------|--------------|------------------------------------------------|
| 1      | CNN          | Convolutional neural network                   |
| 2      | IUCN         | International union for conservation of nature |
| 3      | GPU          | Graphical Processing Unit                      |
| 4      | GUI          | Graphical User Interface                       |
| 5      | UML          | Unified Modeling Language                      |
| 6      | HCL          | Hardware Compatibility List                    |
| 7      | CPU          | Central Processing Unit                        |
| 8      | RAM          | Random Access Memory                           |
| 9      | API          | Application Programming Interface              |
| 10     | IDLE         | Integrated Devolpment and Learning Environment |
| 11     | QA           | Quality Assurance                              |

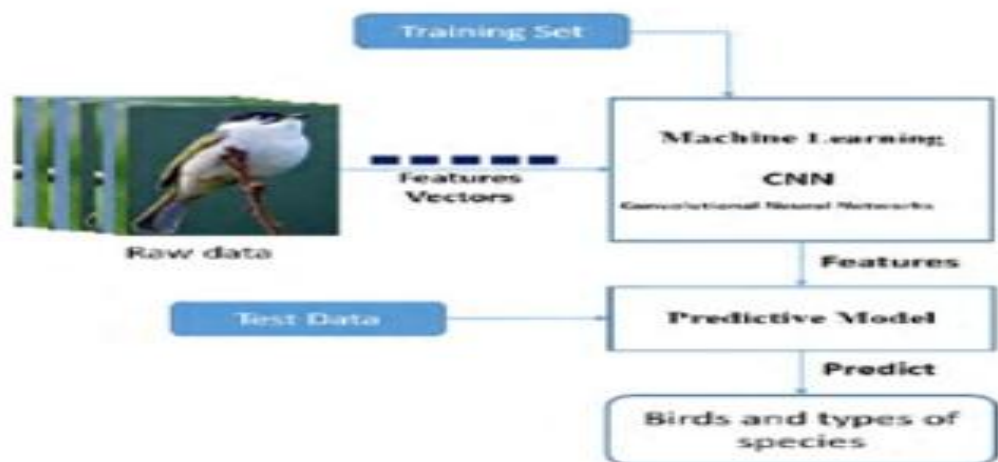
## **ABSTRACT**

In this work, an automatic bird species recognition system has been developed and methods for their identification has been investigated. Automatic identification of bird sounds without physical intervention has been a formidable and onerous endeavor for significant research on the taxonomy and various other sub fields of ornithology. In this paper, a two-stage identification process is employed. The first stage involved construction of an ideal dataset which incorporated all the sound recordings of different bird species. Subsequently, the sound clips were subjected to various sound preprocessing techniques like pre-emphasis, framing, silence removal and reconstruction. Spectrograms were generated for each reconstructed sound clip. The second stage involved deploying a neural network to which the spectrograms were provided as input. Based on the input features, the Convolutional Neural Network (CNN) classifies the sound clip and recognizes the bird species. A Real time implementation model was also designed and executed for the above described system.

**Keywords:** Convolution, Neural Network, Histogram, Audio processing, Noise removal, Deep Learning, Classification.

## 1. INTRODUCTION

According to the International Union for Conservation of Nature (IUCN), there are nearly 10,000 known species of birds dispersed among a vast range of ecosystems, from the rainforests of Brazil to the icy shores of Antarctica. These species exhibit amazing diversity in terms of behavior and morphology and are quintessential for the normal functioning of an ecosystem. But this magnificent biological diversity has been threatened the recent human activities which range from intrusion into their habitats to complete annihilation of their habitats and this coupled with natural phenomenon like global warming and climate change has driven many species to extinction. It is estimated that nearly 1,370 species are threatened with extinction which amounts to nearly 13% of the entire bird population. Although a number of bird species are commonly seen, they are difficult to identify by the people. The aim of this study is to develop automated techniques for identifying bird species from their sounds. Automatic identification of bird calls from continuous recordings gathered from the environment would be significant addition to the research methodology in ornithology and biology, in general. Often these recordings are clipped or contain noise due to which reliable methods of automated techniques have to be used instead of manual conventional methods. Manual inspections of the spectrograms are often error prone and usually the techniques are esoteric in nature and involves multiple experts which makes it unreliable, hence there is a need for automated systems. There is a significant commercial potential for such systems because bird watching is a popular hobby in many countries. Substantial international programs are also invigorating the ventures into the area of bioacoustics signal processing and pattern recognition.



**Fig 1.1: process flow diagram**

The behaviour and population trends of birds have recently become a serious problem. Birds' quick responses to ecological changes enable humans to distinguish between many earthly life forms. However, compiling and acquiring information about bird species is a labor-intensive and expensive process that demands a lot of human effort. In such a situation, a strong framework that will enable the massive preparation of bird data and serve as a significant tool for scientists, legislative agencies, etc. is needed. In this sense, determining which bird species a particular image of belongs to plays a vital role in determining which categories it belongs to.

Bird species identification is determining which category a particular species of bird falls under using a photograph. Images, audio, and videos can all be used to identify different bird species. It is possible to identify birds by listening for their distinctive sounds thanks to an audio processing technique. However, handling of such data becomes increasingly complicated because to the mixed sounds in the condition, such as creepy crawlies, real-world items, and so forth. Images typically aid in information discovery more so than sounds or recordings. In this perspective, utilizing an image rather than voice or video to categorize birds is preferred. Bird species identification is a difficult issue for both humans and computational processes that perform such a task automatically. But, for more accuracy, despite of the above stated complications, we proceeded to audio based classification.

## 2. AIM & SCOPE

### 2.1 PROBLEM DESCRIPTION

To develop a bird species identification system that utilizes neural networks and audio signal processing techniques to accurately identify different species of birds based on their unique vocalizations. The system will process the audio signal using various techniques such as spectrogram analysis, feature extraction, and machine learning algorithms. The developed model should be able to identify multiple bird species accurately and in real-time. The project aims to address the challenge of accurately identifying bird species in the field, which can be difficult and time-consuming for human observers, and to provide a tool for researchers and conservationists to monitor bird populations and habitats more efficiently.

### 2.2 EXISTING SYSTEM

A significant improvement to the study methods in ornithology and biology in general would be the automatic identification of bird cries from continuous recordings collected from the environment. These recordings frequently have clipping or noise, thus trustworthy automated approaches must be employed rather than manual, traditional methods. There is a need for automated systems because manual checks of the spectrograms are frequently erroneous and typically need numerous experts using esoteric methodologies. Such systems have a substantial commercial potential because bird watching is a well-liked hobby in many nations. Significant international projects are also boosting the bioacoustics signal processing and pattern recognition businesses.

#### ***2.2.1 Disadvantages of existing system:***

1. Manual inspections of the spectrograms are often error prone and usually the techniques are esoteric in nature and involves multiple experts which makes it unreliable, hence there is a need for automated systems.

### 2.3 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ECONOMICAL FEASIBILITY
- TECHNICAL FEASIBILITY
- SOCIAL FEASIBILITY

### ***2.3.1 Economical Feasibility***

Economic feasibility is a type of feasibility study that evaluates whether a proposed project or idea is financially viable and economically profitable. It assesses the financial aspects of the project, including the initial investment, operating costs, revenue projections, and potential returns on investment. The economic feasibility study involves analyzing the cost-benefit of the project and estimating the potential financial outcomes. It considers various factors such as market demand, competition, pricing, and regulatory requirements to determine the profitability of the project. The goal of the economic feasibility study is to determine whether the proposed project or idea is economically viable and sustainable in the long run. It helps project managers and stakeholders to assess the financial risks and benefits of the project, identify potential sources of revenue, and develop a realistic financial plan for the project's implementation. The economic feasibility study is an essential tool for decision-making, and it helps stakeholders to make informed decisions about the project's financial viability and potential profitability.

### ***2.3.2 Technical Feasibility***

Technical feasibility is a type of feasibility study that focuses on assessing whether a proposed project or idea can be developed and implemented using existing technology and available resources. The study examines the technical requirements of the project and evaluates the feasibility of meeting those requirements. The technical feasibility study involves a detailed analysis of the project's technical specifications, including hardware, software, and other equipment requirements. It also evaluates the availability of necessary resources such as skilled personnel, raw materials, and technology infrastructure. The goal of the technical feasibility study is to identify any technical barriers or limitations that may prevent the successful implementation of the project. This study helps project managers and stakeholders to make informed decisions regarding the feasibility of the project and to plan for the necessary resources and technical infrastructure required to complete the project successfully.

### ***2.3.3 Social Feasibility***

Social feasibility is a type of feasibility study that examines the potential social impacts of a proposed project or idea. It assesses how the project will affect the community, stakeholders, and society at large. The social feasibility study involves analyzing various factors such as the project's impact on employment, public health, education, cultural heritage, and social norms. It also assesses the potential social risks and benefits of the project and identifies any potential social challenges that may arise. The goal of the social feasibility study is to ensure that the project aligns with the community's social values and expectations. It helps project managers and stakeholders to identify potential social issues and develop strategies to mitigate any negative impacts on the community. It also helps to build community support for the project by engaging with stakeholders and addressing their concerns and needs. The social feasibility study is an essential tool for decision-making, and it helps stakeholders to make informed decisions about the project's social impact and its acceptability in the community.

### 3. REVIEW OF LITERATURE

#### A. Identification of Bird Species from Their Singing:

One of the enigmatic phenomena that we are learning more about as time goes on is biodiversity. The BIOTOPE organization has launched an initiative to aid in the process of comprehending nature. The group's open platform will bring scholars together to extract information by identifying different bird species from their calls or songs. The fundamental concept of this work is to recognize different bird species by their songs or calls. The research discussed in this article investigates an automated system for classifying various bird species based on their songs and calls. This will make it easier to catalogue and identify new species or subspecies. Additionally, it will support research on bird behaviour and ecology and help with population surveillance. In this work, we put into practise a system that can find different bird species from audio recordings. According on the ROC performance analysis of the proposed system, Random Decision Tree may produce 0.96 area under the curve.

#### B. FFT Based Automatic Species Identification Improvement with 4- layer Neural Network:

An automated approach for identifying species has been devised in this paper. Recorded data was automatically split, analyzed, had features removed, and recognized. A 4-layer neural network-based feature quantity technique based on FFT and frequency band power derivative is provided. Wild bird species identification using a 4-layer neural network based on reliable data has been tested, and the results have shown promise.

#### C. Feature set comparison for automatic bird species identification:

In order to identify the species of a bird from its audio recorded song, an issue known as automated bird species identification must be solved. Given that it is an indirect, non-invasive method of evaluation, this is an ingenious way to keep track of ecosystem biodiversity. This study compares various feature sets that describe the audio characteristics of the audio stream in various ways with machine learning methods like neural networks, and SVM. Three different bird species' recorded songs are used as the subject of experiments. The experimental results demonstrate that it is possible to achieve highly encouraging results in the automated bird species identification. The results compare the performance of the feature sets and several classifiers.



D. Detecting bird sounds in a complex acoustic environment and application to bio-acoustic monitoring:

Bird population trends are a crucial indication for environmental protection, but determining these trends requires a lot of time and human effort. Automated approaches for the detection of bird vocalizations are now conceivable through enormous advancements in audio signal processing and pattern recognition in present times. These tactics can be used to support population size censuses. A study that examined the viability of automatic bird song detection in conjunction with bird monitoring was highlighted in this task. Describing new methods for the automatic habitat mapping of the vocalizations of two bird species that are in danger of extinction was given high importance. These techniques work by identifying temporal trends in a certain frequency band that are specific characteristic of the species. The noise that is present in real-world audio scenarios is specifically suppressed. Insights demonstrate that high recognition rates with a manageable number of false positive detection are still feasible under real-world recording situations.

E. Automated sound recording and analysis techniques for bird surveys and conservation:

Automated sound recording and analysis of bird sounds need to be used more and developed further. Since birds are essential to the health of ecosystems, science and conservation efforts would substantially benefit from methods that make avian monitoring more effective and precise. We give an overview of the popular methods used in software to automatically identify and categorize avian sound, as well as an overview of the hardware approaches to automated sound recording. A hardware interface for a scheduling timer to drive a standalone commercial recorder, a programmable recording device, and a single board computer are three examples of the three basic kinds of hardware solutions we compare and contrast for automating sound recording. We also give examples of the two basic methods using wave guides and tiny arrays of microphone elements for enhancing microphone performance for automated recorders. We propose five fundamental sound fragment kinds of bird sounds for automated sound analysis and describe various methods to automatically recognize and categorize avian sounds down to the species level, as well as their drawbacks. Along with a range of categorization techniques for those features, a variety of features to measure for the various call kinds are offered. Their general performance, as well as the monitoring and conservation initiatives they help with, are highlighted.

#### F. Deep Learning Based Audio Classifier for Bird Species

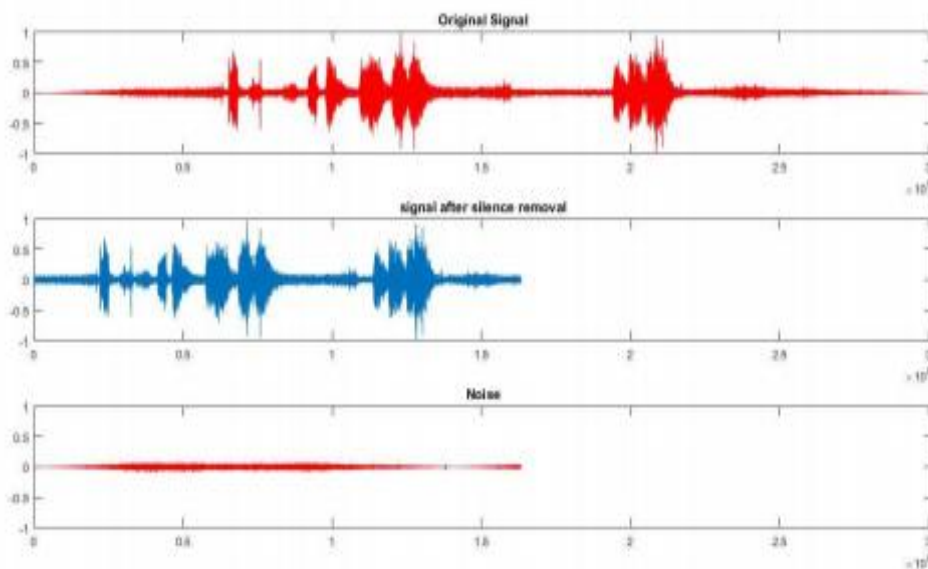
Providing a machine learning method to recognize the bird species based on the audio clips was the sole purpose of this work. Deep Residual Neural Network which is a state of art technology was used. Use of data augmentation improvised the effectiveness of this method. The model produced a decent accuracy of 80% when trained on the dataset containing 30 sound classes of audio clips.

## 4. PROPOSED SOLUTION

A technique has been proposed which involves the use of sound processing and convolutional neural networks to automate the entire process of bird sound identification. The first stage involves the creation of a database consisting of all the sound recordings. Subsequently, these recordings are subjected to sound pre-processing techniques like preemphasis, framing, silence removal and reconstruction. Spectrograms are generated for the sound clips and these spectrograms were given as input to a CNN which was trained on a GPU. A real time implementation model was designed for the trained CNN model. A Graphical User Interface (GUI) was also designed for the system. An application can be designed in the future and deployed for mobile devices, which would enable users to use their smartphones as handheld devices for prediction and analysis of bird sounds.

### 4.1 Methodology

In this paper, a method for automating the entire process of identifying bird sounds using sound processing and convolutional neural networks is proposed. The initial step entails compiling all of the sound recordings into a database. Then, sound pre-processing methods like pre-emphasis, framing, silence removal, and reconstruction are applied to these recordings.



**Fig 4.1.1: silence removal**

### ***Pre-emphasis***

It is a technique to process signals that involves improvising the frequency components of an audio signal. It amplifies high audio components and diminishes the intensity of low frequency components.

### ***Framing***

This process divides audio signal into small and manageable parts. It assists in processing the signals with ease. But, very short duration of frames results in higher computational burden.

### ***Silence removal***

It is an approach to detect and remove the periods of silence removal from an audio signal. One common approach followed is to set a threshold frequency. If any signal is detected below this frequency, it is removed from the sample. This threshold is set based on the averages of all frequencies in the signal.

### ***Reconstruction***

This process involves digitally encoded audio signal back into its original wave form. It generates continuous wave form from the digital signal which matches the nearest frequency.

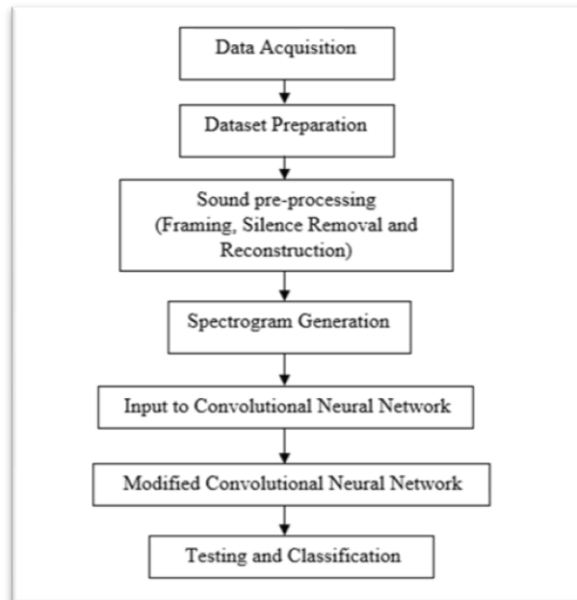
For each sound clip, spectrograms are produced, and these spectrograms are fed into a CNN that has been trained on a GPU. For the trained CNN model, a real-time implementation model was developed.

### ***Advantages:***

- The system has a 98% accuracy rate when classifying bird species based on the spectrogram image created from their sounds.
- This precision was attained by taking into account both human voices and avian sounds.
- By fine-tuning the performance settings, the precision can be increased even more.

### ***Process flow***

- *Data exploration*: using this module we will load data into system
- *Processing*: Using the module we will read data for processing
- *Splitting data into train & test*: using this module data will be divided into train & test
- *Model generation*: Building the model Alexnet with CNN.
- *Predict accuracy*: Using this module will get we are displaying accuracy, precision, recall and FSCORE
- *User input*: Using this module will give input for prediction
- *Prediction*: final predicted displayed.



***Fig 4.1.2: flow chart***

## 5. SYSTEM REQUIREMENTS

### **5.1 Hardware Requirements**

The most common set of requirements defined by any operating system or software application is the physical computer resources, also known as hardware. A hardware requirements list is often accompanied by a hardware compatibility list (HCL), especially in case of operating systems. An HCL lists tested, compatible, and sometimes incompatible hardware devices for a particular operating system or application.

*Architecture* – All computer operating systems are designed for a particular computer architecture. Most software applications are limited to particular operating systems running on particular architectures. Although architecture-independent operating systems and applications exist, most need to be recompiled to run on a new architecture. See also a list of common operating systems and their supporting architectures.

*Processing power* – The power of the central processing unit (CPU) is a fundamental system requirement for any software. Most software running on x86 architecture define processing power as the model and the clock speed of the CPU. Many other features of a CPU that influence its speed and power, like bus speed, cache, and MIPS are often ignored. This definition of power is often erroneous, as AMD Athlon and Intel Pentium CPUs at similar clock speed often have different throughput speeds. Intel Pentium CPUs have enjoyed a considerable degree of popularity, and are often mentioned in this category.

*Memory* – All software, when run, resides in the random access memory (RAM) of a computer. Memory requirements are defined after considering demands of the application, operating system, supporting software and files, and other running processes. Optimal performance of other unrelated software running on a multi-tasking computer system is also considered when defining this requirement.

*Secondary storage* – Hard-disk requirements vary, depending on the size of software installation, temporary files created and maintained while installing or running the software, and possible use of swap space (if RAM is insufficient).

*Display adapter* – Software requiring a better than average computer graphics display, like graphics editors and high-end games, often define high-end display adapters in the system requirements.

*Peripherals* – Some software applications need to make extensive and/or special use of some peripherals, demanding the higher performance or functionality of such peripherals. Such peripherals include CD-ROM drives, keyboards, pointing devices, network devices, etc.

- Operating System : Windows Only
- Processor : i5 and above
- Ram : 4gb and above
- Hard Disk : 50 GB

## **5.2 Software requirements**

Software requirements deal with defining software resource requirements and prerequisites that need to be installed on a computer to provide optimal functioning of an application. These requirements or prerequisites are generally not included in the software installation package and need to be installed separately before the software is installed.

*Platform* – In computing, a platform describes some sort of framework, either in hardware or software, which allows software to run. Typical platforms include a computer's architecture, operating system, or programming languages and their runtime libraries.

Operating system is one of the first requirements mentioned when defining system requirements (software). Software may not be compatible with different versions of same line of operating systems, although some measure of backward compatibility is often maintained. For example, most software designed for Microsoft Windows XP does not run on Microsoft Windows 98, although the converse is not always true. Similarly, software designed using newer features of Linux Kernel v2.6 generally does not run or compile properly (or at all) on Linux distributions using Kernel v2.2 or v2.4.

*APIs and drivers* – Software making extensive use of special hardware devices, like high-end display adapters, needs special API or newer device drivers. A good example is DirectX, which is a collection of APIs for handling tasks related to multimedia, especially game programming, on Microsoft platforms.

*Web browser* – Most web applications and software depending heavily on Internet technologies make use of the default browser installed on system. Microsoft Internet Explorer is a frequent choice of software running on Microsoft Windows, which makes use of ActiveX controls, despite their vulnerabilities.

- Visual Studio Community Version
- Nodejs ( Version 12.3.1)
- Python IDLE ( Python 3.7 )

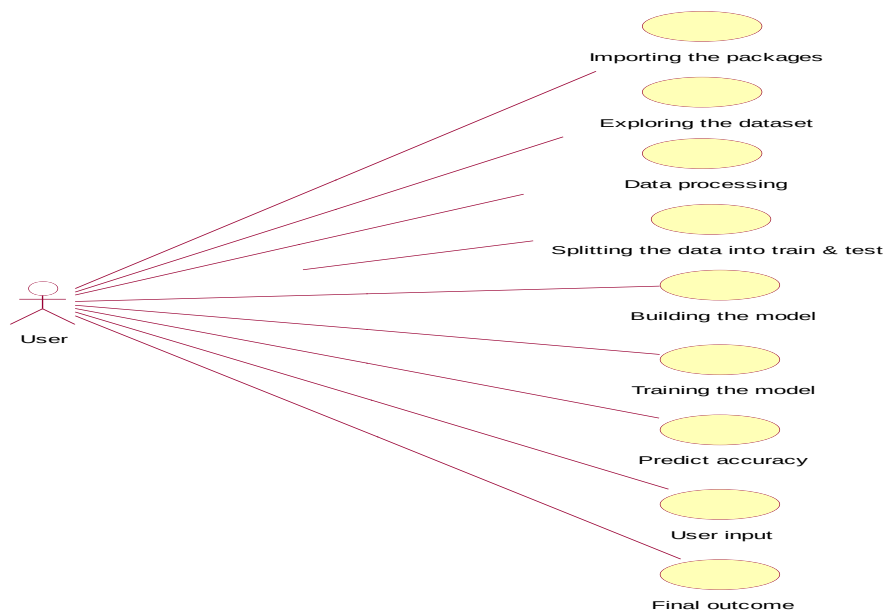


## 6. SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It involves the creation of a detailed plan or blueprint for the construction of the system. System design is a crucial phase in the software development life cycle, and it follows the requirements gathering and analysis phase.

### 6.1 Use case diagram

A use case diagram is a type of Unified Modeling Language (UML) diagram used to describe the functionality of a system from the user's perspective. It is a graphical representation of the interactions between the system and the users or external systems. The use case diagram consists of actors, use cases, and relationships between them. Actors represent the roles played by users or external systems that interact with the system being modeled. Use cases represent the actions or functionalities that the system performs to achieve specific goals. Relationships between actors and use cases describe the interactions between them. The use case diagram is useful for visualizing the system's behavior and understanding the requirements from the user's perspective. It helps to identify the actors and use cases required for the system, their relationships, and the system's overall functionality. It also provides a basis for further analysis and design of the system.



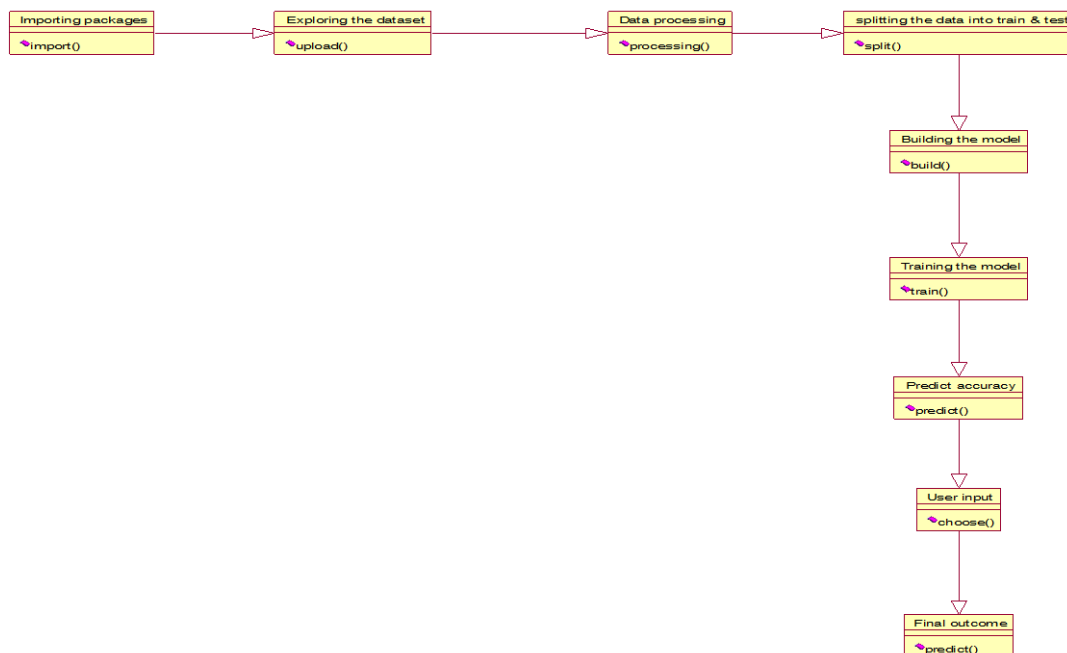
**Fig 6.1.1: usecase diagram**

## 6.2 Class diagram

A class diagram is a type of UML (Unified Modeling Language) diagram used to represent the static structure of a system. It shows the classes, interfaces, and their relationships to each other. Class diagrams consist of classes, interfaces, associations, dependencies, and inheritance relationships. Here are some common elements of a class diagram:

- *Class*: A class is a blueprint or template for creating objects that have similar attributes and behavior.
- *Interface*: An interface defines a set of methods that a class must implement.
- *Association*: An association represents a relationship between two classes, indicating that they are related in some way.
- *Dependency*: A dependency indicates that one class depends on another class.
- *Inheritance*: Inheritance is a relationship between a superclass and a subclass, indicating that the subclass inherits attributes and behaviors from the superclass.

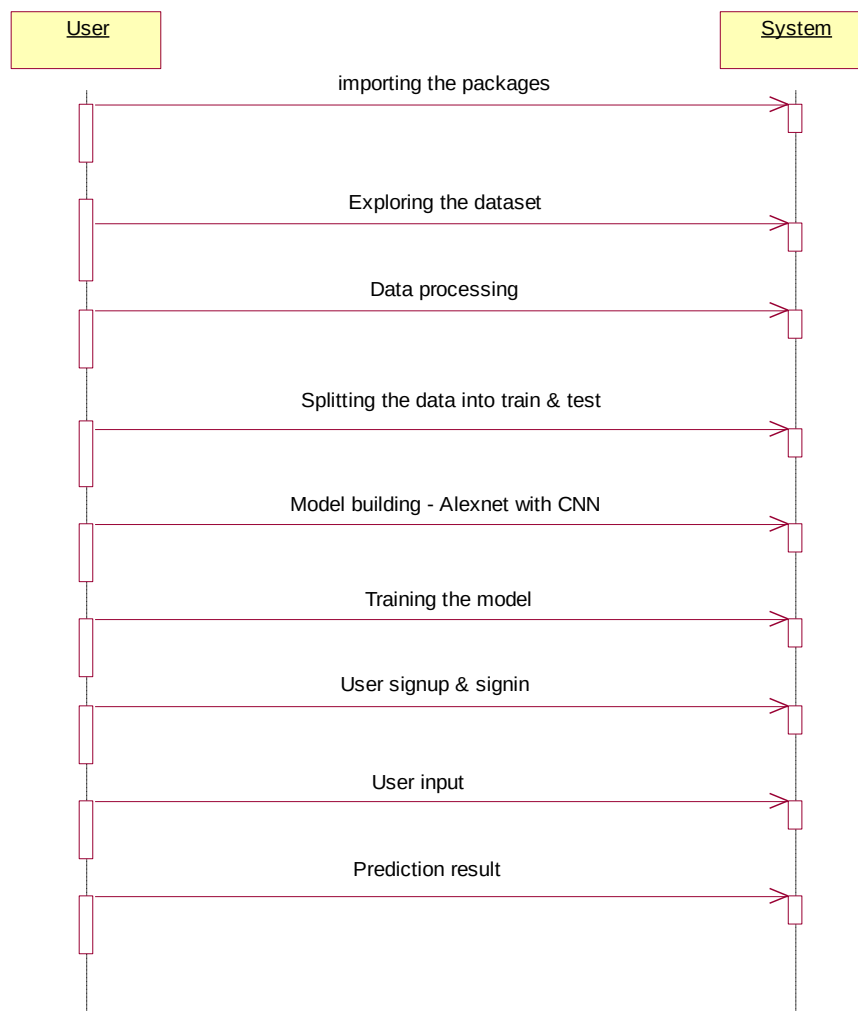
Class diagrams are useful in software engineering to represent the static structure of a system and its components. They help to understand the relationships and dependencies between different classes and their attributes and methods. They also assist in creating a robust and maintainable software system.



**Fig 6.2.1: class diagram**

### 6.3 Sequence diagram

A sequence diagram is a type of UML (Unified Modeling Language) diagram used to represent the interaction between objects in a system over time. It shows the sequence of messages exchanged between the objects and the order in which they occur. Sequence diagrams consist of vertical lines called lifelines, which represent the objects or actors involved in the sequence. The horizontal arrows connecting the lifelines represent the messages exchanged between the objects. Each message is labeled with the name of the message and its parameters, if any. The sequence of messages is shown on the vertical axis of the diagram, from top to bottom. Here are the common elements of a sequence diagram:



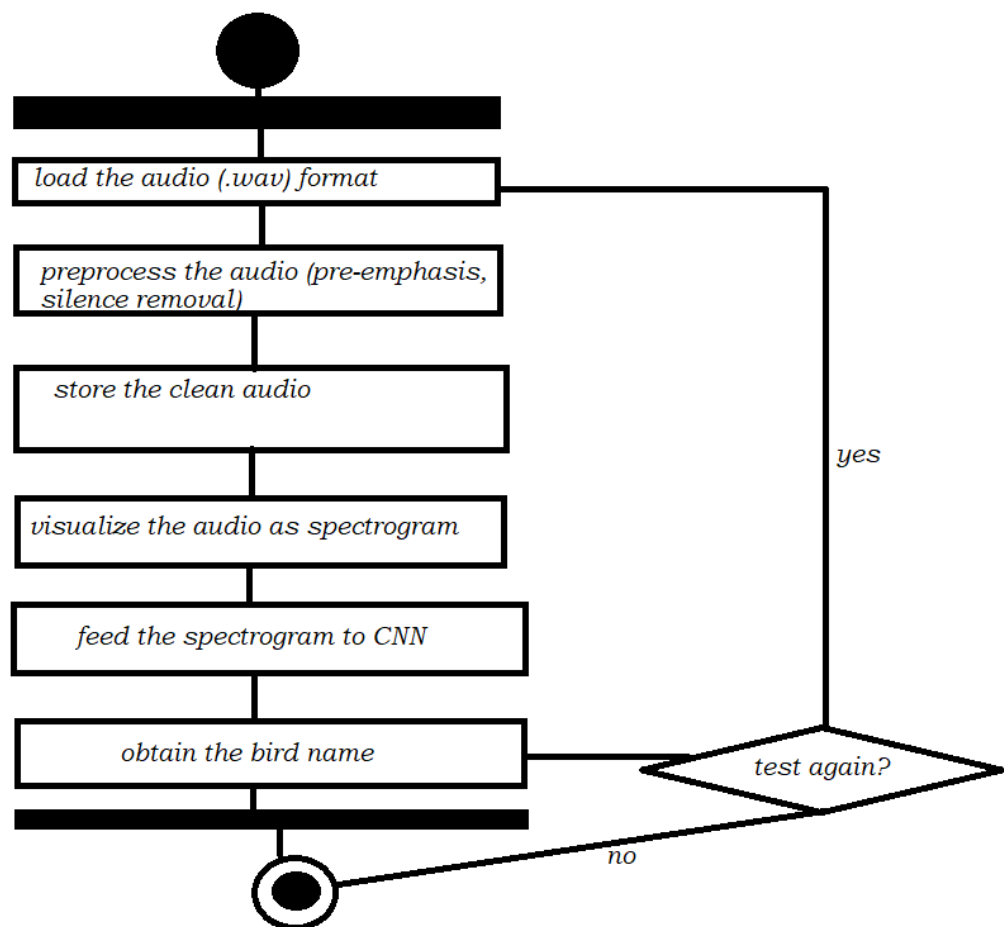
**Fig 6.3.1: sequence diagram**

- Lifeline: This represents an object or actor in the system and is depicted as a vertical line.

- **Activation:** This represents the period during which an object is performing an action, and it is depicted as a horizontal bar on the lifeline.
- **Message:** This represents a communication between two objects or actors and is depicted as an arrow between the lifelines. It can be synchronous or asynchronous.
- **Return Message:** This represents the response of an object to a message it received, and it is depicted as a dotted arrow.
- **Self-Message:** This represents a message that an object sends to itself, and it is depicted as a loopback arrow.

Sequence diagrams are used in software engineering to model the interaction between objects or actors in a system and to analyze and optimize system behavior. They are particularly useful for modeling complex systems and identifying potential issues with system performance and reliability.

#### 6.4 Statechart diagram



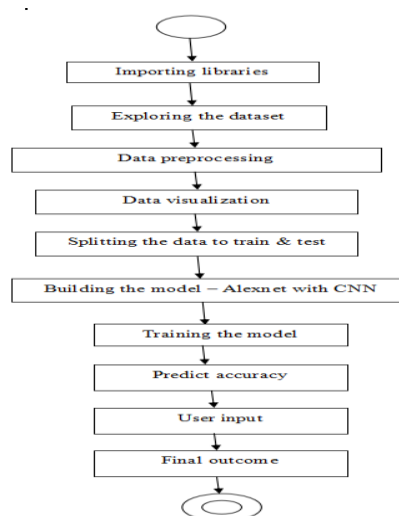
##### 6.4.1 state chart diagram

It is a behavioral diagram. it portrays the system's behavior utilizing finite state transitions. It is also known as the State-charts diagram. It models the dynamic behavior of a class in response to external stimuli.

### 6.5 Activity diagram

An activity diagram is a type of UML (Unified Modeling Language) diagram used to represent the flow of activities or processes in a system. It is a graphical representation of the steps and activities involved in a particular process or workflow. Activity diagrams consist of a series of nodes and arrows that represent activities and the relationships between them. Nodes are used to represent activities or actions, while arrows are used to show the flow of control or the sequence of activities. Here are the common elements of an activity diagram:

- Initial Node: This represents the starting point of the activity diagram.
- Action or Activity: This represents a specific task or action that needs to be performed.
- Decision or Branching: This represents a point in the process where a decision needs to be made, and the process will follow one of the several available paths depending on the decision.
- Fork or Join: These nodes are used to divide the flow of the process into multiple parallel paths or to join them back together.
- End Node: This represents the completion of the process or workflow.

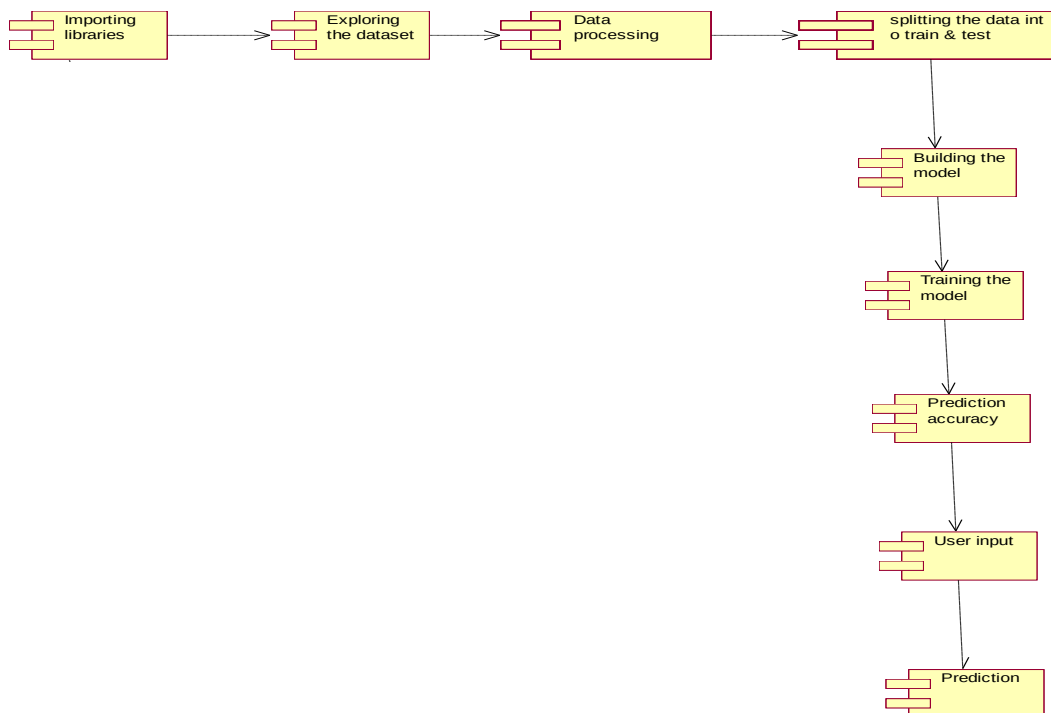


**Fig 6.5.1: activity diagram**

Activity diagrams are used in software engineering and other industries to model complex business processes, system workflows, and software behavior. They are helpful in analyzing and optimizing processes and identifying potential bottlenecks or inefficiencies.

## 6.6 Component diagram

A component diagram is a type of UML (Unified Modeling Language) diagram used to visualize the structural relationships and dependencies among the components of a system. It shows the physical and logical components of the system and the connections between them. Components are the building blocks of a system and represent the implementation units that provide specific functionality. The component diagram helps to identify the dependencies between components, the interfaces they expose, and the relationships they have with other components in the system. Here are the common elements of a component diagram:



**fig 6.5.1: component diagram**

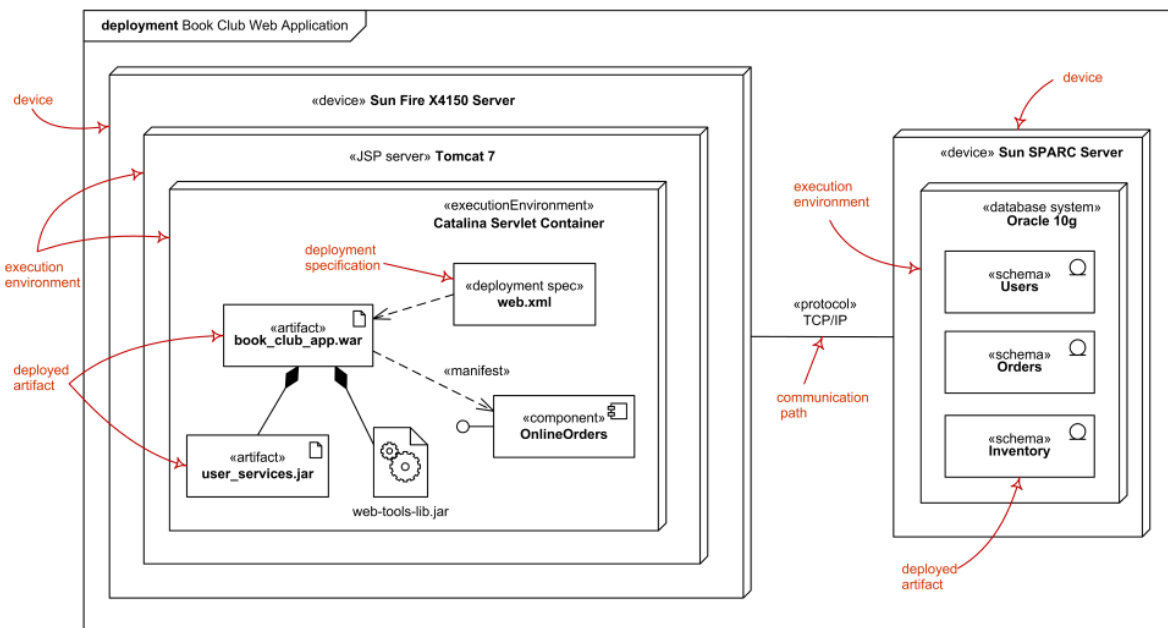
- Component: This represents a software module or physical element of the system.
- Interface: This represents a contract or agreement between the components that specifies how they communicate with each other.
- Dependency: This shows the relationship between two components where one component depends on the other for its implementation.

- Association: This shows the relationship between two components where they are loosely coupled and have a weaker dependency.
- Package: This represents a collection of related components that are organized together.

Component diagrams are used in software engineering to design and document the software architecture of a system. They help to visualize the components and their relationships, making it easier to understand the system's structure and design. They are also helpful in identifying potential bottlenecks or issues in the system's architecture.

## 6.7 Deployment diagram

A deployment diagram is a type of UML (Unified Modeling Language) diagram that shows the physical deployment of a system or software application in a hardware environment. It is used to model the distribution of components, modules, and nodes in a system and the communication between them. Deployment diagrams consist of nodes, components, and connectors.



### 6.7.1 deployment diagram

Here are the common elements of a deployment diagram:

- Nodes: Nodes represent hardware devices or software execution environments, such as servers, workstations, or mobile devices.

- **Components:** Components represent the software modules or components that are deployed on the nodes, such as applications, web services, or databases.
- **Artifacts:** Artifacts represent the files or data that are used by the components.
- **Connectors:** Connectors represent the communication paths between the nodes and components.
- **Deployment Specifications:** Deployment specifications specify the configuration and deployment settings for the components and nodes.

Deployment diagrams are used to model and plan the deployment of software systems in various environments, such as local networks, cloud-based environments, and mobile devices. They are useful for visualizing the physical layout of a system and identifying potential deployment issues and requirements.



## 7. IMPLEMENTATION

### 7.1 TOOLS USED

#### ***PYTHON LANGUAGE:***

Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together. Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance. Python supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms, and can be freely distributed. Often, programmers fall in love with Python because of the increased productivity it provides. Since there is no compilation step, the edit-test-debug cycle is incredibly fast. Debugging Python programs is easy: a bug or bad input will never cause a segmentation fault. Instead, when the interpreter discovers an error, it raises an exception. When the program doesn't catch the exception, the interpreter prints a stack trace. A source level debugger allows inspection of local and global variables, evaluation of arbitrary expressions, setting breakpoints, stepping through the code a line at a time, and so on. The debugger is written in Python itself, testifying to Python's introspective power. On the other hand, often the quickest way to debug a program is to add a few print statements to the source: the fast edit-test-debug cycle makes this simple approach very effective.

Python is a dynamic, high-level, free open source, and interpreted programming language. It supports object-oriented programming as well as procedural-oriented programming. In Python, we don't need to declare the type of variable because it is a dynamically typed language. For example, `x = 10` Here, `x` can be anything such as String, int, etc.

#### *Features in Python:*

There are many features in Python, some of which are discussed below as follows:

##### *1. Free and Open Source*

Python language is freely available at the official website and you can download it from the given download link below click on the Download Python keyword. Download

Python Since it is open-source, this means that source code is also available to the public. So you can download it, use it as well as share it.

## *2. Easy to code*

Python is a high-level programming language. Python is very easy to learn the language as compared to other languages like C, C#, Javascript, Java, etc. It is very easy to code in the Python language and anybody can learn Python basics in a few hours or days. It is also a developer-friendly language.

## *3. Easy to Read*

As you will see, learning Python is quite simple. As was already established, Python's syntax is really straightforward. The code block is defined by the indentations rather than by semicolons or brackets.

## *4. Object-Oriented Language*

One of the key features of Python is Object-Oriented programming. Python supports object-oriented language and concepts of classes, object encapsulation, etc.

## *5. GUI Programming Support*

Graphical User interfaces can be made using a module such as PyQt5, PyQt4, wxPython, or Tk in python. PyQt5 is the most popular option for creating graphical apps with Python.

## *6. High-Level Language*

Python is a high-level language. When we write programs in Python, we do not need to remember the system architecture, nor do we need to manage the memory.

## *7. Extensible feature*

Python is an Extensible language. We can write some Python code into C or C++ language and also we can compile that code in C/C++ language.

## *S. Easy to Debug*

Excellent information for mistake tracing. You will be able to quickly identify and correct the majority of your program's issues once you understand how to interpret Python's error traces. Simply by glancing at the code, you can determine what it is designed to perform.

### *S. Python is a Portable language*

Python language is also a portable language. For example, if we have Python code for windows and if we want to run this code on other platforms such as Linux, Unix, and Mac then we do not need to change it, we can run this code on any platform.

### *10. Python is an Integrated language*

Python is also an Integrated language because we can easily integrate Python with other languages like C, C++, etc.

### *11. Interpreted Language:*

Python is an Interpreted Language because Python code is executed line by line at a time. Like other languages C, C++, Java, etc. there is no need to compile Python code this makes it easier to debug our code. The source code of Python is converted into an immediate form called bytecode.

### *12. Large Standard Library*

Python has a large standard library that provides a rich set of modules and functions so you do not have to write your own code for every single thing. There are many libraries present in Python such as regular expressions, unit-testing, web browsers, etc.

### *13. Dynamically Typed Language*

Python is a dynamically-typed language. That means the type (for example- int, double, long, etc.) for a variable is decided at run time not in advance because of this feature we don't need to specify the type of variable.

### *14. Frontend and backend development*

With a new project py script, you can run and write Python codes in HTML with the help of some simple tags <py-script>, <py-env>, etc. This will help you do frontend development work in Python like javascript. Backend is the strong forte of Python it's extensively used for this work cause of its frameworks like Django and Flask.

### *15. Allocating Memory Dynamically*

In Python, the variable data type does not need to be specified. The memory is automatically allocated to a variable at runtime when it is given a value. Developers do not need to write `int y = 18` if the integer value 15 is set to y. You may just type `y=18`.

### ***LIBRARIES/PACKGES :-***

#### *Tensorflow*

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine learning applications such as neural networks. It is used for both research and production at Google. TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

#### *Numpy*

Numpy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays. It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated (broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, Numpy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using Numpy which allows Numpy to seamlessly and speedily integrate with a wide variety of databases.

#### *Pandas*

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and

analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

### *Matplotlib*

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and Ipython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery. For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with Ipython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

### *Scikit – learn*

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

## 7.2 ALGORITHMS

### **7.2.1 Modules:**

- *Data exploration:* using this module we will load data into system
- *Processing:* Using the module we will read data for processing
- *Splitting data into train & test:* using this module data will be divided into train & test
- *Model generation:* Building the model Alexnet with CNN.
- *Predict accuracy:* Using this module will get we are displaying accuracy, precision, recall and FSCORE
- *User input:* Using this module will give input for prediction
- *Prediction:* final predicted displayed

### 7.2.2 Algorithms:

*ALEXNET with CNN:* A convolutional neural network (CNN) is a specific type of artificial neural network that uses perceptrons, a machine learning unit algorithm, for supervised learning, to analyze data. CNNs apply to image processing, natural language processing and other kinds of cognitive tasks. AlexNet was the first convolutional network which used GPU to boost performance. 1. AlexNet architecture consists of 5 convolutional layers, 3 max-pooling layers, 2 normalization layers, 2 fully connected layers, and 1 softmax layer. AlexNet is a leading architecture for any object-detection task and may have huge applications in the computer vision sector of artificial intelligence problems. In the future, AlexNet may be adopted more than CNNs for image tasks.

In order to identify or predict the species of bird, we are using the AlexNet CNN model, and in order to do so, we are using two different modules. The first module is used to read bird audio files, after which audio framing and noise removal are applied, and finally spectrogram features are extracted from the processed audio file. Applying the pre-trained CNN model from AlexNet on the extracted features in the second module to create a bird species recognition model. using the datasets of XENO CANTO bird audio files, which incorporates noise signals, to train the model author.

A particular kind of artificial neural network called a convolutional neural network (CNN) analyses data using perceptrons, a technique for supervised learning. CNNs can be used for processing images, natural language, and other cognitive tasks. The first convolutional network to employ a GPU to improve performance was AlexNet. Five convolutional layers (three max-pooling layers, two normalization layers), added 3 extra layers with (9,9), (7,7) and (6,6) number of neurons respectively, two fully connected layers, and one Softmax layer make up the AlexNet architecture.

For any object-detection task, AlexNet is a top architecture, and it may find extensive use in computer vision-related artificial intelligence issues. In the future, AlexNet might be used for picture jobs more frequently than CNNs.

A multi-layer perceptron with a specific architecture for identifying two-dimensional picture data is the convolution neural network algorithm. An input layer, a convolution layer, a sample layer, and an output layer are among its four layers. The sample layer and convolution layer in a deep network architecture could have more than one. Convolution neural network techniques do not have the same limitations as the Boltzmann machine in

that each neuron just needs to sense the local portion of the image rather than the entire image, as is the case with the Boltzmann machine. Additionally, each neuron's parameters are specified to be the same, including the sharing of weights and the use of identical convolution kernels for the deconvolution picture for each neuron.

The central elements of CNN are the local receptive field, weight sharing, and time- or space-based sub sampling with the aim of feature extraction and training parameter reduction. The benefit of the CNN algorithm is that it learns intuitively from training data rather than explicitly extracting features. The network can learn in parallel and become less complex because the same neuron weights are on the surface of the feature mapping, utilizing a sub-sampling framework based on deformation displacement, scale, and time resilience. Network topology and input data may go well together. It offers special benefits for picture processing. These actions are part of the Convolution Neural Network:

#### *Convolution Layer:*

The convolutional layer is the core constructing block of a CNN. The convolution layer comprises a set of independent feature detectors. Each Feature map is independently convoluted with the images.

#### *Pooling Layer:*

The pooling layer feature is to progressively reduce the spatial size of the illustration to reduce the wide variety of parameters and computation in the network. The pooling layer operates on each function map independently. The approaches used in pooling are:

- Max Pooling
- Mean Pooling
- Sum Pooling

#### *Fully Connected Layer:*

Neurons in the fully connected layer have full connections to all activations inside the preceding layer. In this, the output obtained from max pooling is converted to a one dimensional array and that should be the input layer and the process continues the same as the ANN model.

### 7.2.3 Sample code:

```
import logging

import torch
import torch.nn as nn
from torch import cuda
from torch.autograd import Variable
from torch.utils.data import DataLoader, Dataset

import torchvision
import torchvision.datasets as dset
import torchvision.transforms as transforms
import torchvision.utils
from PIL import Image

import torch.nn.functional as F

import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import numpy as np
import random
from collections import Counter

from custom_transform2D import CustomResize
from custom_transform2D import CustomToTensor

from AD_Dataset import AD_Dataset
from AD_Standard_2DslicesData import AD_Standard_2DslicesData
from AD_Standard_2DrandomSlicesData import AD_Standard_2DrandomSlicesData
from AD_Standard_2DtestingSlices import AD_Standard_2DtestingSlices

from AlexNet2D import alexnet

logging.basicConfig(
    format='%(asctime)s %(levelname)s: %(message)s',
    datefmt='%Y-%m-%d %H:%M:%S', level=logging.INFO)

parser = argparse.ArgumentParser(description="Starter code for JHU CS661 Computer Vision HW3.")

parser.add_argument("-load",
                    help="Load saved network weights.")
parser.add_argument("-save", default="AlexNet",
                    help="Save network weights.")
parser.add_argument("-augmentation", default=True, type=bool,
                    help="Save network weights.")
parser.add_argument("-epochs", default=20, type=int,
```



```

        help="Epochs through the data. (default=20)")
parser.add_argument("-learning_rate", "-lr", default=1e-3, type=float,
                    help="Learning rate of the optimization. (default=0.01)")
parser.add_argument('-momentum', default=0.9, type=float, metavar='M',
                    help='momentum')
parser.add_argument('-weight-decay', '-wd', default=1e-4, type=float,
                    metavar='W', help='weight decay (default: 1e-4)')
parser.add_argument("-estop", default=1e-2, type=float,
                    help="Early stopping criteria on the development set.
(default=1e-2)")
parser.add_argument("-batch_size", default=1, type=int,
                    help="Batch size for training. (default=1)")
parser.add_argument("-optimizer", default="Adam", choices=["SGD", "Adadelata",
"Adam"],
                    help="Optimizer of choice for training. (default=Adam)")
parser.add_argument("-gpid", default=[0], nargs='+', type=int,
                    help="ID of gpu device to use. Empty implies cpu usage.")

```

# feel free to add more arguments as you need

```

def main(options):
    # Path configuration
    TRAINING_PATH = 'train_2classes.txt'
    TESTING_PATH = 'test_2classes.txt'
    IMG_PATH = './Image'

    trg_size = (224, 224)

    transformations = transforms.Compose([CustomResize(trg_size),
                                         CustomToTensor()
                                         ])

    dset_train = AD_Standard_2DRandomSlicesData(IMG_PATH, TRAINING_PATH,
transformations)
    dset_test = AD_Standard_2DslicesData(IMG_PATH, TESTING_PATH, transformations)

    # Use argument load to distinguish training and testing
    if options.load is None:
        train_loader = DataLoader(dset_train,
                                batch_size=options.batch_size,
                                shuffle=True,
                                num_workers=4,
                                drop_last=True
                                )
    else:
        # Only shuffle the data when doing training
        train_loader = DataLoader(dset_train,

```

```

        batch_size=options.batch_size,
        shuffle=False,
        num_workers=4,
        drop_last=True
    )

test_loader = DataLoader(dset_test,
                        batch_size=options.batch_size,
                        shuffle=False,
                        num_workers=4,
                        drop_last=True
    )

use_cuda = (len(options.gpuid) >= 1)
if options.gpuid:
    cuda.set_device(options.gpuid[0])

# Initial the model
model = alexnet(pretrained=True)
# model.load_state_dict(torch.load(options.load))

if use_cuda > 0:
    model.cuda()
else:
    model.cpu()

# Binary cross-entropy loss
# criterion = torch.nn.CrossEntropyLoss()
criterion = torch.nn.NLLLoss()

lr = options.learning_rate
optimizer = eval("torch.optim." + options.optimizer)(filter(lambda x:
x.requires_grad, model.parameters()), lr)

best_accuracy = float("-inf")

train_loss_f = open("train_loss.txt", "w")
test_acu_f = open("test_accuracy.txt", "w")

for epoch_i in range(options.epochs):

    logging.info("At {0}-th epoch.".format(epoch_i))
    train_loss, correct_cnt = train(model, train_loader, use_cuda, criterion,
optimizer, train_loss_f)
    # each instance in one batch has 3 views
    train_avg_loss = train_loss / (len(dset_train) * 3 / options.batch_size)
    train_avg_acu = float(correct_cnt) / (len(dset_train) * 3)

```

```

        logging.info(
            "Average training loss is {0:.5f} at the end of epoch
{1}".format(train_avg_loss.data[0], epoch_i))
        logging.info("Average training accuracy is {0:.5f} at the end of epoch
{1}".format(train_avg_acu, epoch_i))

        correct_cnt = validate(model, test_loader, use_cuda, criterion)
        dev_avg_acu = float(correct_cnt) / len(dset_test)
        logging.info("Average validation accuracy is {0:.5f} at the end of epoch
{1}".format(dev_avg_acu, epoch_i))

        # write validation accuracy to file
        test_acu_f.write("{0:.5f}\n".format(dev_avg_acu))

        if dev_avg_acu > best_accuracy:
            best_accuracy = dev_avg_acu
            torch.save(model.state_dict(), open(options.save, 'wb'))

    train_loss_f.close()
    test_acu_f.close()

def train(model, train_loader, use_cuda, criterion, optimizer, train_loss_f):
    # main training loop
    train_loss = 0.0
    correct_cnt = 0.0
    model.train()
    for it, train_data in enumerate(train_loader):
        for data_dic in train_data:
            if use_cuda:
                imgs, labels = Variable(data_dic['image']).cuda(),
Variable(data_dic['label']).cuda()
            else:
                imgs, labels = Variable(data_dic['image']),
Variable(data_dic['label'])
            integer_encoded = labels.data.cpu().numpy()
            # target should be LongTensor in loss function
            ground_truth = Variable(torch.from_numpy(integer_encoded)).long()
            if use_cuda:
                ground_truth = ground_truth.cuda()
            train_output = model(imgs)
            _, predict = train_output.topk(1)
            loss = criterion(train_output, ground_truth)
            train_loss += loss
            correct_this_batch = (predict.squeeze(1) ==
ground_truth).sum().float()
            correct_cnt += correct_this_batch
        accuracy = float(correct_this_batch) / len(ground_truth)

```

```

        logging.info("batch {0} training loss is : {1:.5f}".format(it,
loss.data[0]))
        logging.info("batch {0} training accuracy is : {1:.5f}".format(it,
accuracy))

        # write the training loss to file
        train_loss_f.write("{0:.5f}\n".format(loss.data[0]))

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    return train_loss, correct_cnt

def validate(model, test_loader, use_cuda, criterion):
    # validation - this is a crude estimation because there might be some
    paddings at the end
    correct_cnt = 0.0
    model.eval()
    for it, test_data in enumerate(test_loader):
        vote = []
        for data_dic in test_data:
            if use_cuda:
                imgs, labels = Variable(data_dic['image'], volatile=True).cuda(),
Variable(data_dic['label'],
volatile=True).cuda()
            else:
                imgs, labels = Variable(data_dic['image'], volatile=True),
Variable(data_dic['label'],
volatile=True)
            test_output = model(imgs)
            _, predict = test_output.topk(1)
            vote.append(predict)

        vote = torch.cat(vote, 1)
        final_vote, _ = torch.mode(vote, 1)
        ground_truth = test_data[0]['label']
        correct_this_batch = (final_vote.cpu().data == ground_truth).sum()
        correct_cnt += correct_this_batch
        accuracy = float(correct_this_batch) / len(ground_truth)

        logging.info("batch {0} dev accuracy is : {1:.5f}".format(it, accuracy))

    return correct_cnt

```

```

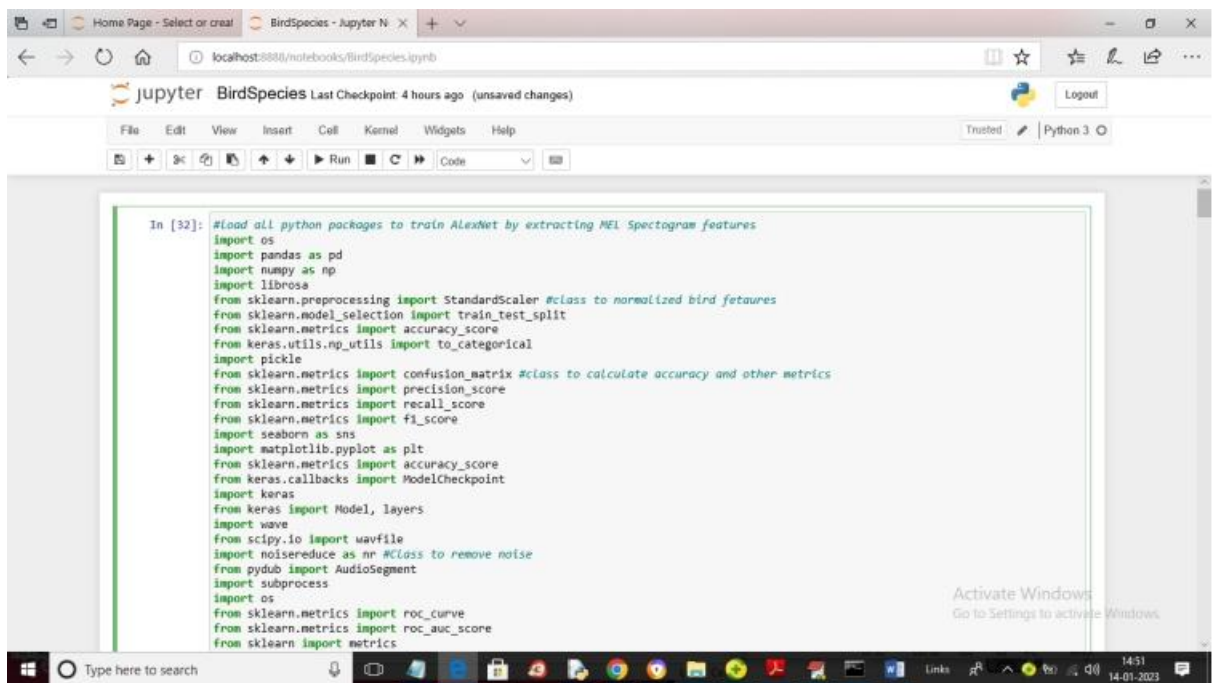
def show_plot(points):
    plt.figure()
    fig, ax = plt.subplots()
    loc = ticker.MultipleLocator(base=0.2) # put ticks at regular intervals
    ax.yaxis.set_major_locator(loc)
    plt.plot(points)

if __name__ == "__main__":
    ret = parser.parse_known_args()
    options = ret[0]
    if ret[1]:
        logging.warning("unknown arguments:
{0}".format(parser.parse_known_args()[1]))
    main(options)

```

## 8.RESULTS

In this paper author is using AlexNet CNN model to identify or predict bird species and to predict species author is using two different modules where first module is used to read bird audio files and then apply audio framing and noise removal and then extract spectrogram features from processed audio file. In second module applying AlexNet pre-trained CNN model on extracted features to build bird Species identification model. To train model author using XENO CANTO bird audio files dataset and this audio contains noise signals and after removing silence and noise we are saving processed audio files inside 'NoiseCleanAudio' file folder. To implement this project we have used JUPYTER NOTEBOOK and below are the output screens with blue colour comments.



```
In [32]: #load all python packages to train AlexNet by extracting MEL Spectrogram features
import os
import pandas as pd
import numpy as np
import librosa
from sklearn.preprocessing import StandardScaler #class to normalized bird features
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from keras.utils.np_utils import to_categorical
import pickle
from sklearn.metrics import confusion_matrix #class to calculate accuracy and other metrics
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import accuracy_score
from keras.callbacks import ModelCheckpoint
import keras
from keras import Model, layers
import wave
from scipy.io import wavfile
import noisereduce as nr #class to remove noise
from pydub import AudioSegment
import subprocess
import os
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn import metrics
```

**Fig 8.1: import packages**

In above screen we are importing require python packages

```

import noisereduce as nr #class to remove noise
from pydub import AudioSegment
import subprocess
import os
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score
from sklearn import metrics

In [33]: path = "audio"
if os.path.exists("NoiseCleanAudio") == False: #function to clean noise and silence from audio
    for root, dirs, directory in os.walk(path): #loop all files
        for j in range(len(directory)):
            name = os.path.basename(root)
            subprocess.call(['ffmpeg', '-i', "audio/"+name+"/"+directory[j], 'test.wav']) #convert mp3 to wav file
            rate, data = wavfile.read("test.wav")
            if data.ndim == 1:
                reduced_noise = nr.reduce_noise(y=data, sr=rate) #now remove noise from the audio
                wavfile.write("NoiseCleanAudio/"+name+"/"+directory[j], rate, reduced_noise) #save noised remove audio file
            if os.path.exists("test.wav") == True:
                os.remove("test.wav")

print()
print("All noise cleaned audio file processed and saved inside NoiseCleanAudio")
print()

All noise cleaned audio file processed and saved inside NoiseCleanAudio

In [34]: def getID(name): #function to get ID of the bird as label from species names
        index = 0
        for i in range(len(labels)):

```

**Fig 8.2: noise cleaning**

In above screen defining function to remove noise from all audio files

```

In [34]: def getID(name): #function to get ID of the bird as label from species names
        index = 0
        for i in range(len(labels)):
            if labels[i] == name:
                index = i
                break
        return index
labels = []
for root, dirs, directory in os.walk("NoiseCleanAudio"): #now loop all files and get labels and then display all birds names
    for j in range(len(directory)):
        name = os.path.basename(root)
        if name not in labels:
            labels.append(name)
print("Bird Species found in dataset")
print()
for i in range(len(labels)):
    print(labels[i])

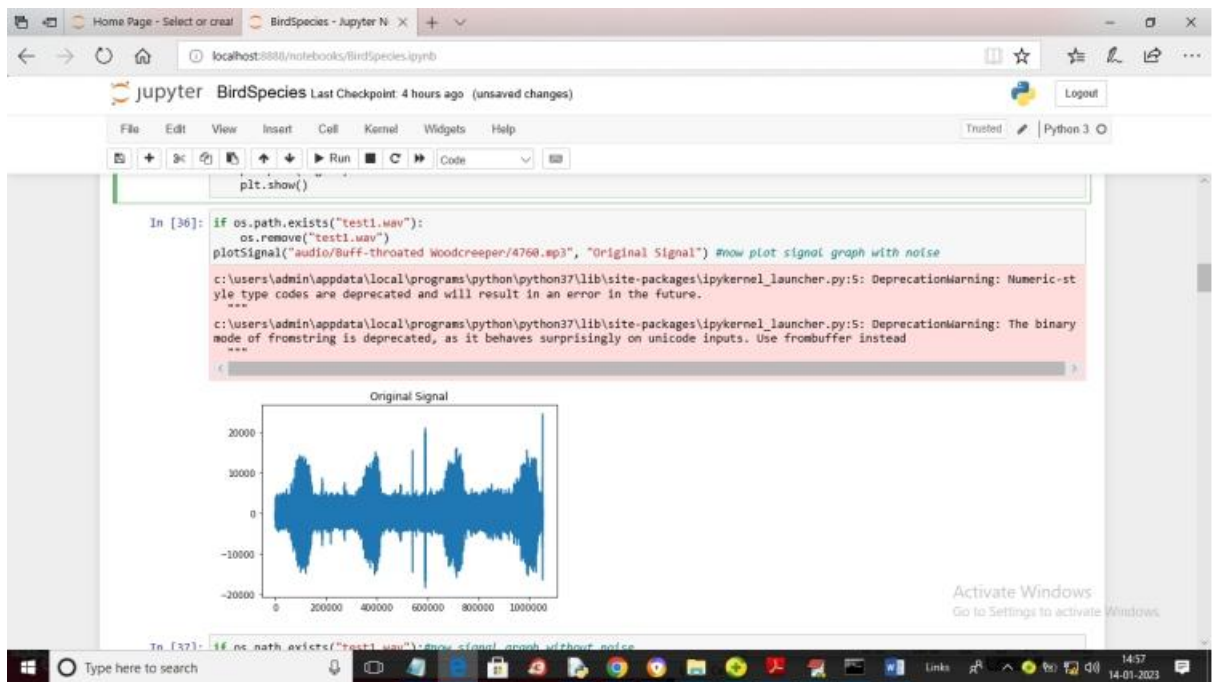
Bird Species found in dataset
Buff-throated Woodcreeper
Lesser Woodcreeper
Olivaceous Woodcreeper
Yellow-olive Flatbill

In [35]: def plotSignal(audio_file, name): #function to plot signal graph
        subprocess.call(['ffmpeg', '-i', audio_file, 'test1.wav'])
        spf = wave.open("test1.wav", "r")
        signal = spf.readframes(-1)
        signal = np.fromstring(signal, "int16")
        plt.figure(1)

```

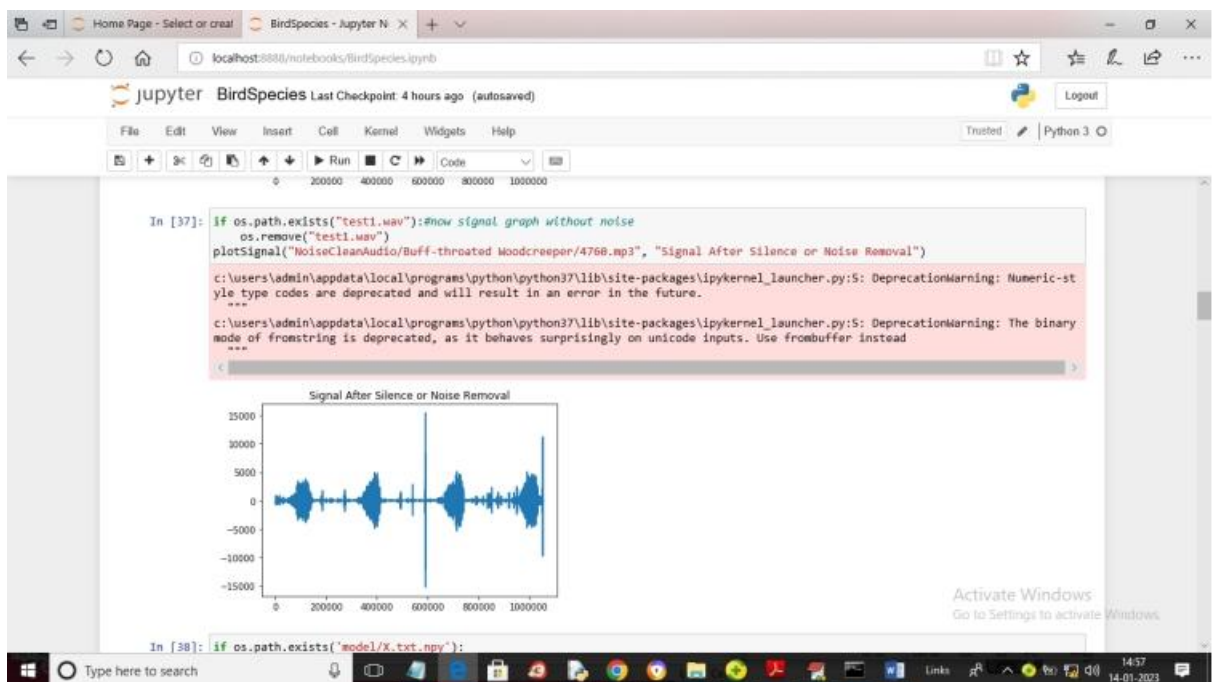
**Fig 8.3: Identifying the species**

In above screen we are finding labels and names of species found in dataset



**fig 8.4: spectrogram of original signal**

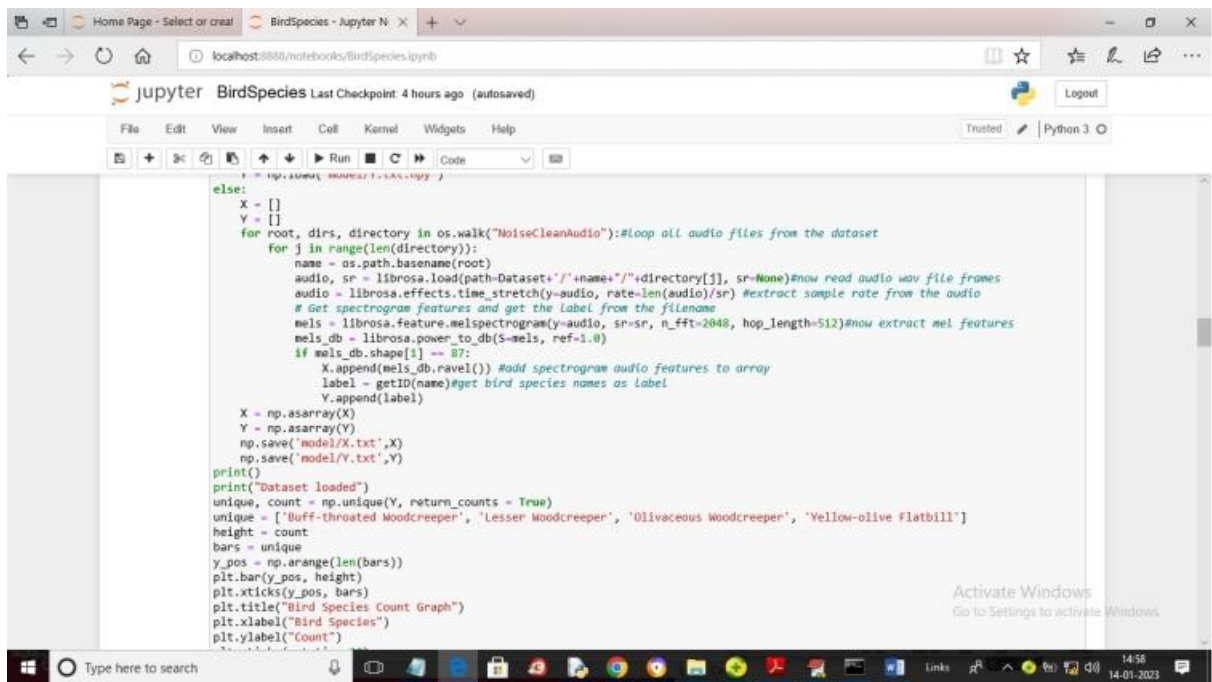
In above screen displaying one audio file with noise



**Fig 8.5: spectrogram of signal after noise removal**

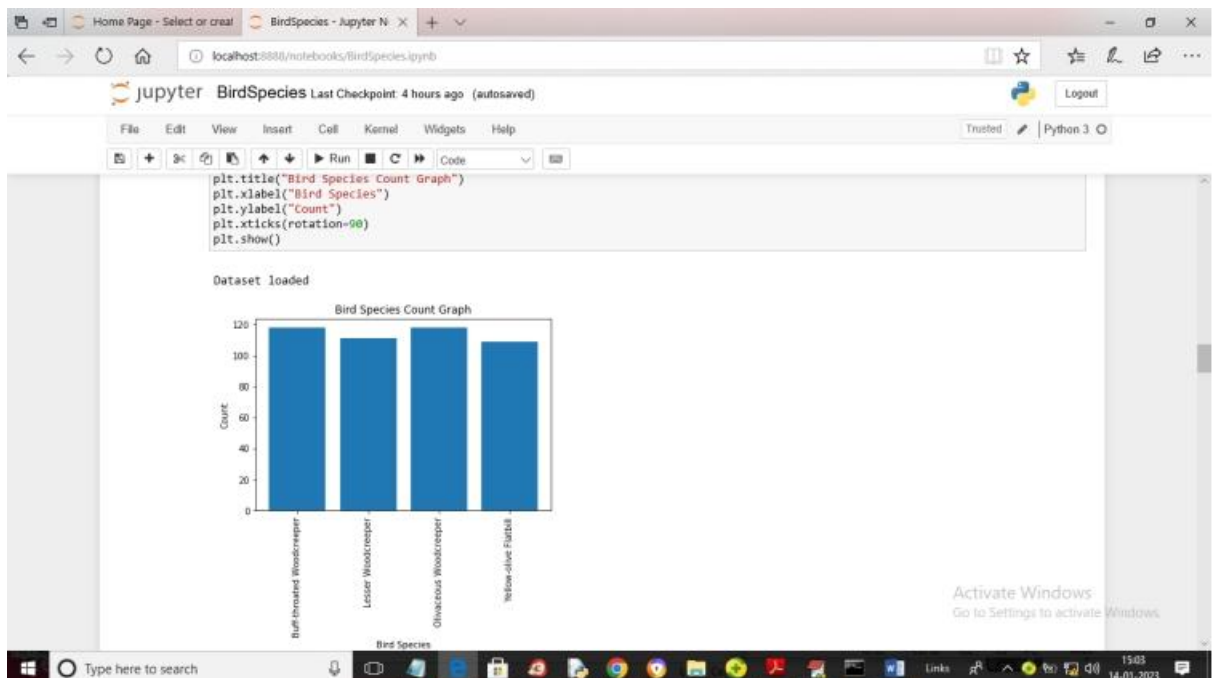
In above screen displaying audio file after removing noise and we can see variance in the graph after noise removal





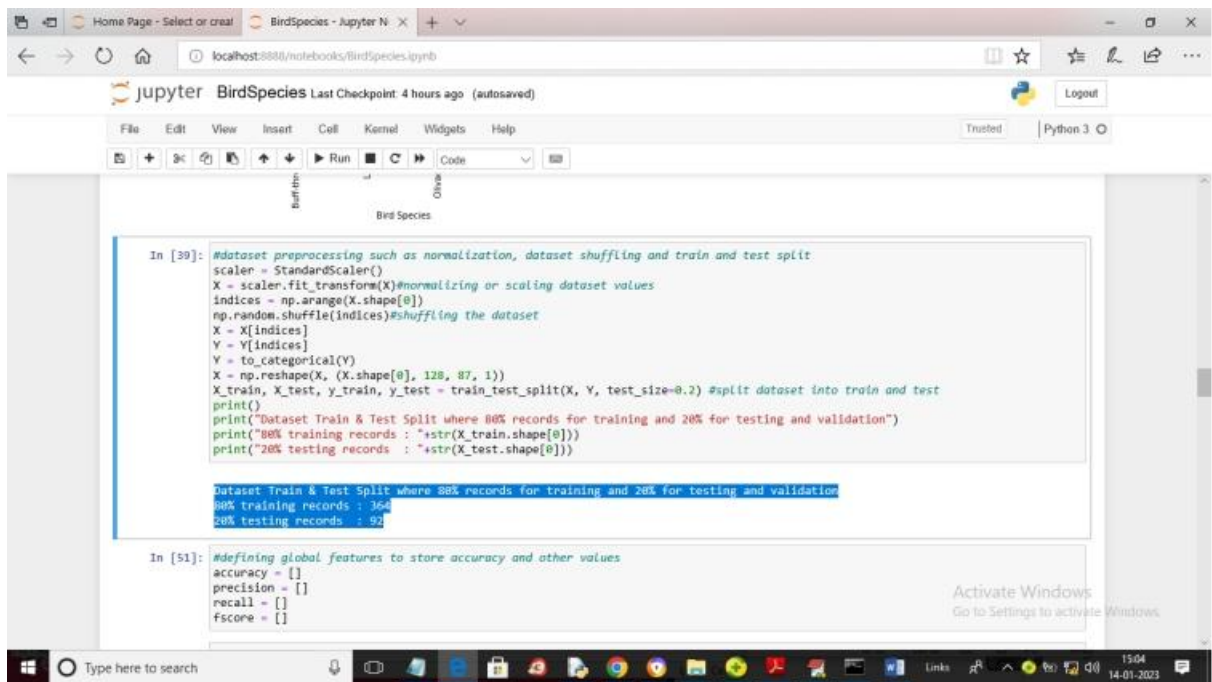
**Fig 8.6: extract spectrogram features**

In above screen we are reading all files and then extracting spectrogram features from bird audio file



**Fig 8.7: data visualization**

In above screen plotting graph of various birds species and their count found in dataset



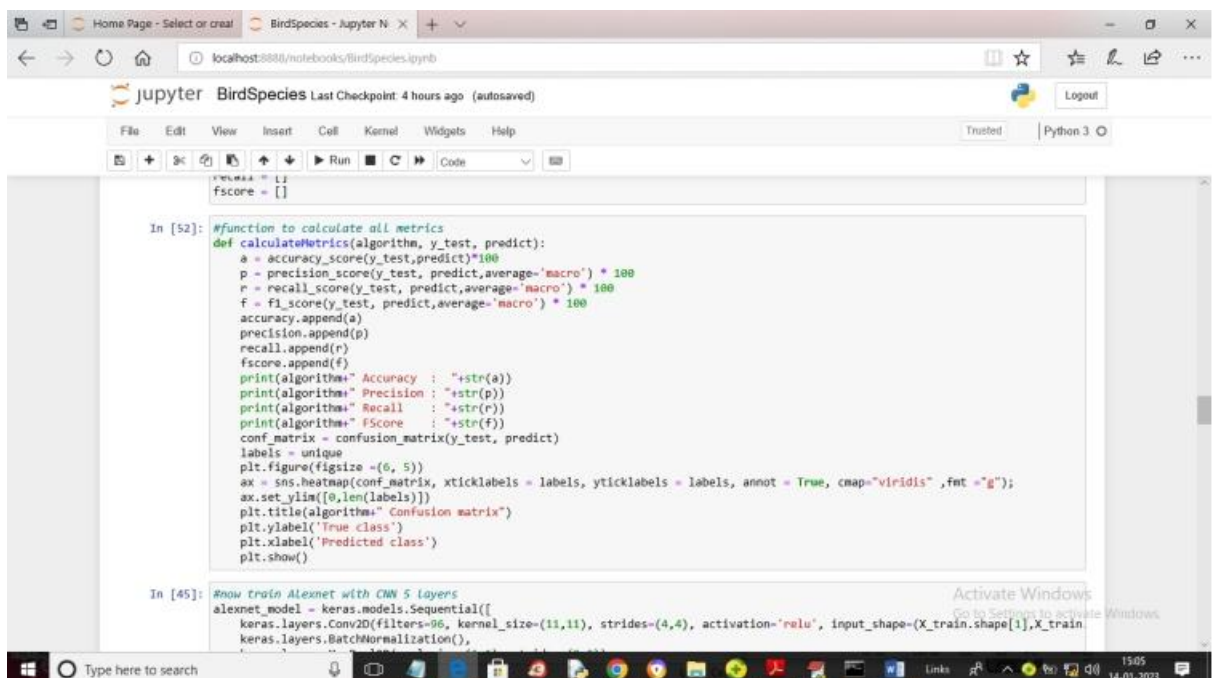
```
In [39]: #dataset preprocessing such as normalization, dataset shuffling and train and test split
scaler = StandardScaler()
X = scaler.fit_transform(X)#normalizing or scaling dataset values
indices = np.arange(X.shape[0])
np.random.shuffle(indices)#shuffling the dataset
X = X[indices]
Y = Y[indices]
Y = to_categorical(Y)
X = np.reshape(X, (X.shape[0], 128, 87, 1))
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset into train and test
print()
print("Dataset Train & Test Split where 80% records for training and 20% for testing and validation")
print("80% training records : "+str(X_train.shape[0]))
print("20% testing records : "+str(X_test.shape[0]))

Dataset Train & Test Split where 80% records for training and 20% for testing and validation
80% training records : 364
20% testing records : 92

In [51]: #defining global features to store accuracy and other values
accuracy = []
precision = []
recall = []
fscore = []
```

**Fig 8.8: split dataset for training and testing**

In above screen we are normalizing, shuffling and splitting dataset in to train and validation and then in blue colour text we can see 80% records are using for training and 20% for validation or testing

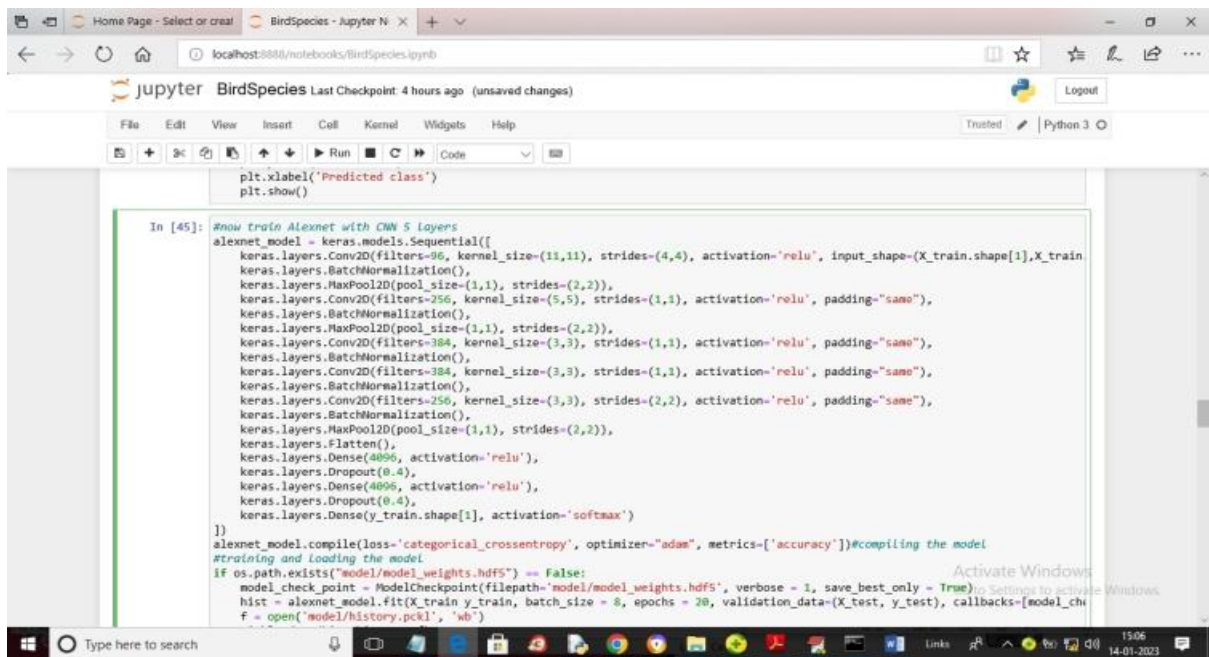


```
In [52]: #function to calculate all metrics
def calculateMetrics(algorithm, y_test, predict):
    a = accuracy_score(y_test, predict)*100
    p = precision_score(y_test, predict, average='macro') * 100
    r = recall_score(y_test, predict, average='macro') * 100
    f = f1_score(y_test, predict, average='macro') * 100
    accuracy.append(a)
    precision.append(p)
    recall.append(r)
    fscore.append(f)
    print(algorithm+" Accuracy : "+str(a))
    print(algorithm+" Precision : "+str(p))
    print(algorithm+" Recall : "+str(r))
    print(algorithm+" FScore : "+str(f))
    conf_matrix = confusion_matrix(y_test, predict)
    labels = unique
    plt.figure(figsize=(6, 5))
    ax = sns.heatmap(conf_matrix, xticklabels = labels, yticklabels = labels, annot = True, cmap="viridis", fmt = "g");
    ax.set_ylim([0, len(labels)])
    plt.title(algorithm+" Confusion matrix")
    plt.ylabel('True class')
    plt.xlabel('Predicted class')
    plt.show()

In [45]: #now train Alexnet with CNN 5 layers
alexnet_model = keras.models.Sequential([
    keras.layers.Conv2D(filters=96, kernel_size=(11,11), strides=(4,4), activation='relu', input_shape=(X_train.shape[1], X_train
```

**Fig 8.9: define performance metrics**

In above screen we are defining function to calculate accuracy and other metrics

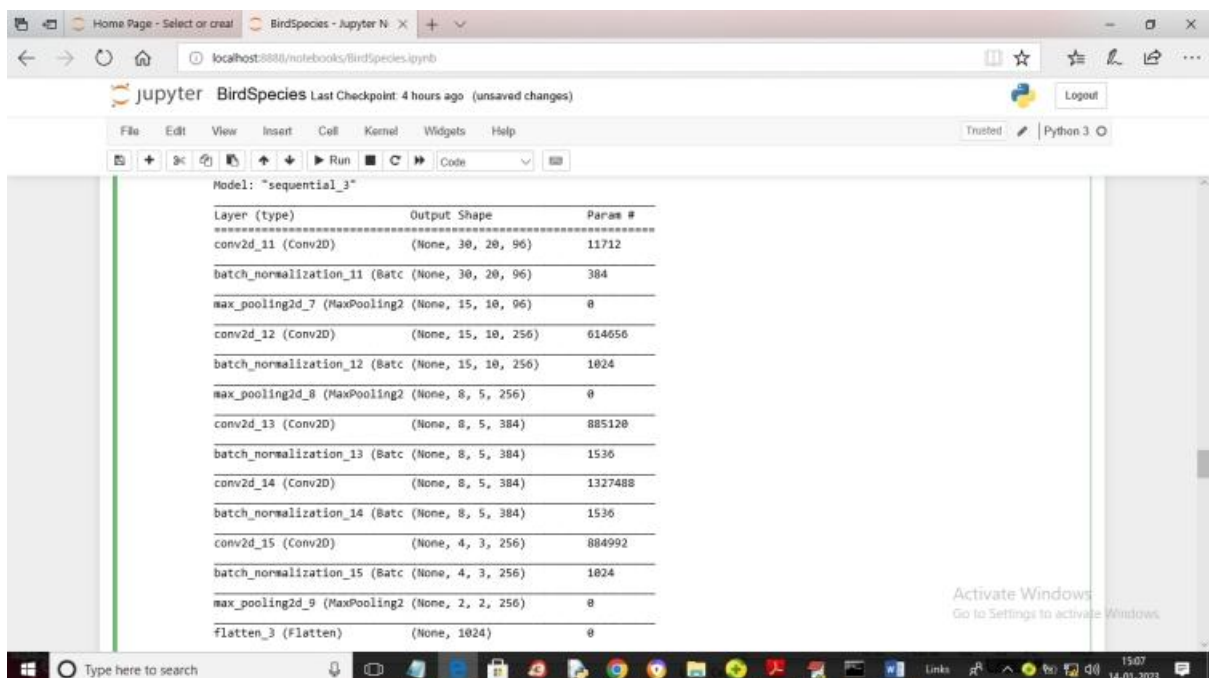


```
plt.xlabel('Predicted class')
plt.show()

In [45]: #now train Alexnet with CNN 5 layers
alexnet_model = keras.models.Sequential([
    keras.layers.Conv2D(filters=96, kernel_size=(11,11), strides=(4,4), activation='relu', input_shape=(X_train.shape[1],X_train
    keras.layers.BatchNormalization(),
    keras.layers.MaxPool2D(pool_size=(2,2), strides=(2,2)),
    keras.layers.Conv2D(filters=256, kernel_size=(5,5), strides=(1,1), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPool2D(pool_size=(2,2), strides=(2,2)),
    keras.layers.Conv2D(filters=384, kernel_size=(3,3), strides=(1,1), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2D(filters=384, kernel_size=(3,3), strides=(1,1), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.Conv2D(filters=256, kernel_size=(3,3), strides=(2,2), activation='relu', padding="same"),
    keras.layers.BatchNormalization(),
    keras.layers.MaxPool2D(pool_size=(2,2), strides=(2,2)),
    keras.layers.Flatten(),
    keras.layers.Dense(4096, activation='relu'),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(4096, activation='relu'),
    keras.layers.Dropout(0.4),
    keras.layers.Dense(y_train.shape[1], activation='softmax')
])
alexnet_model.compile(loss='categorical_crossentropy', optimizer="adam", metrics=['accuracy'])#compiling the model
#training and loading the model
if os.path.exists('model/model_weights.hdf5') == False:
    model_checkpoint = ModelCheckpoint(filepath='model/model_weights.hdf5', verbose = 1, save_best_only = True)
    hist = alexnet_model.fit(X_train, y_train, batch_size = 8, epochs = 20, validation_data=(X_test, y_test), callbacks=[model_ch
    f = open('model/history.pkl', 'wb')
```

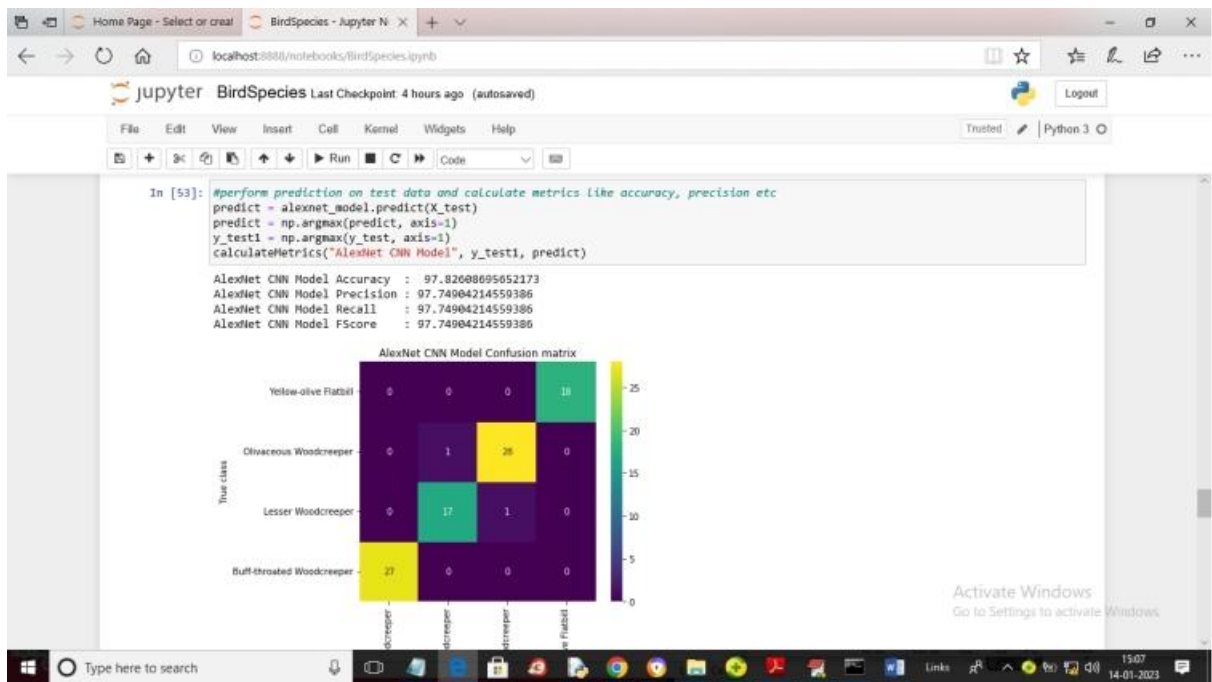
**Fig8.10: build the neural network using keras**

In above screen we are defining AlexNet with CNN 5 layers and then added 3 extra layers for improving training accuracy, and then compiling and training AlexNet model and below is the model details



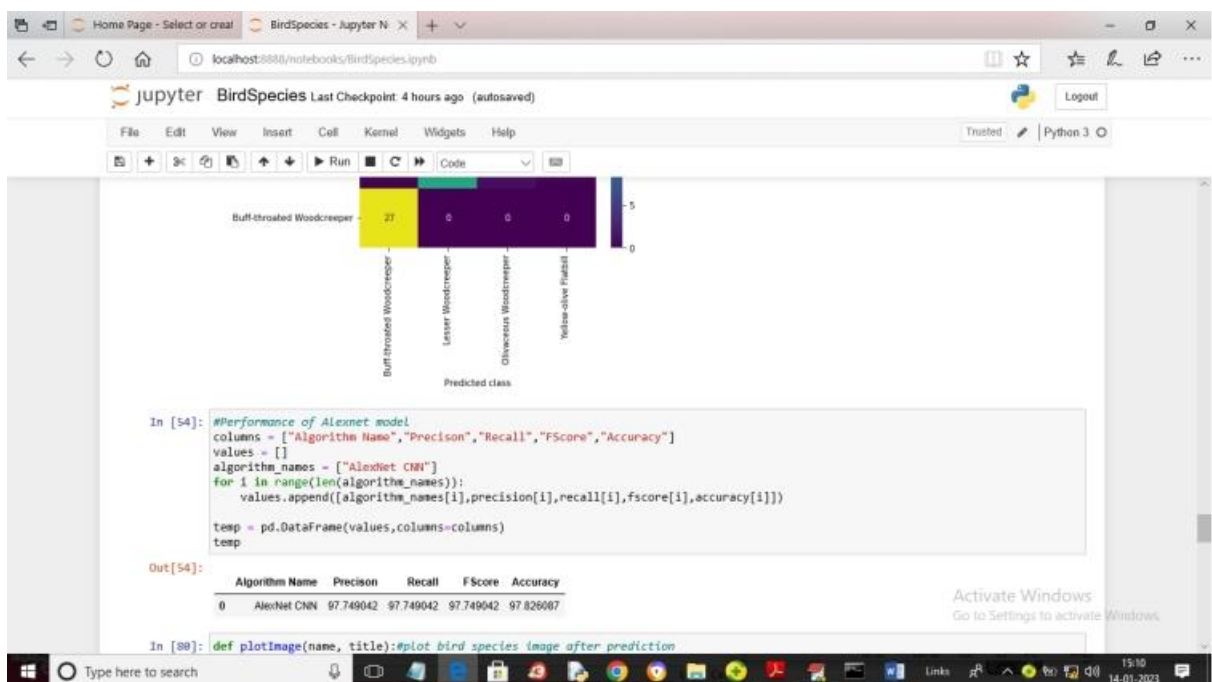
| Layer (type)                                 | Output Shape        | Param # |
|----------------------------------------------|---------------------|---------|
| conv2d_11 (Conv2D)                           | (None, 30, 20, 96)  | 11712   |
| batch_normalization_11 (Batch Normalization) | (None, 30, 20, 96)  | 384     |
| max_pooling2d_7 (MaxPooling2D)               | (None, 15, 10, 96)  | 0       |
| conv2d_12 (Conv2D)                           | (None, 15, 10, 256) | 614656  |
| batch_normalization_12 (Batch Normalization) | (None, 15, 10, 256) | 1024    |
| max_pooling2d_8 (MaxPooling2D)               | (None, 8, 5, 256)   | 0       |
| conv2d_13 (Conv2D)                           | (None, 8, 5, 384)   | 885120  |
| batch_normalization_13 (Batch Normalization) | (None, 8, 5, 384)   | 1536    |
| conv2d_14 (Conv2D)                           | (None, 8, 5, 384)   | 1327488 |
| batch_normalization_14 (Batch Normalization) | (None, 8, 5, 384)   | 1536    |
| conv2d_15 (Conv2D)                           | (None, 4, 3, 256)   | 884992  |
| batch_normalization_15 (Batch Normalization) | (None, 4, 3, 256)   | 1024    |
| max_pooling2d_9 (MaxPooling2D)               | (None, 2, 2, 256)   | 0       |
| flatten_3 (Flatten)                          | (None, 1024)        | 0       |

In above screen we can see AlexNet model with 5 Conv2D layers



**Fig 8.11: confusion matrix**

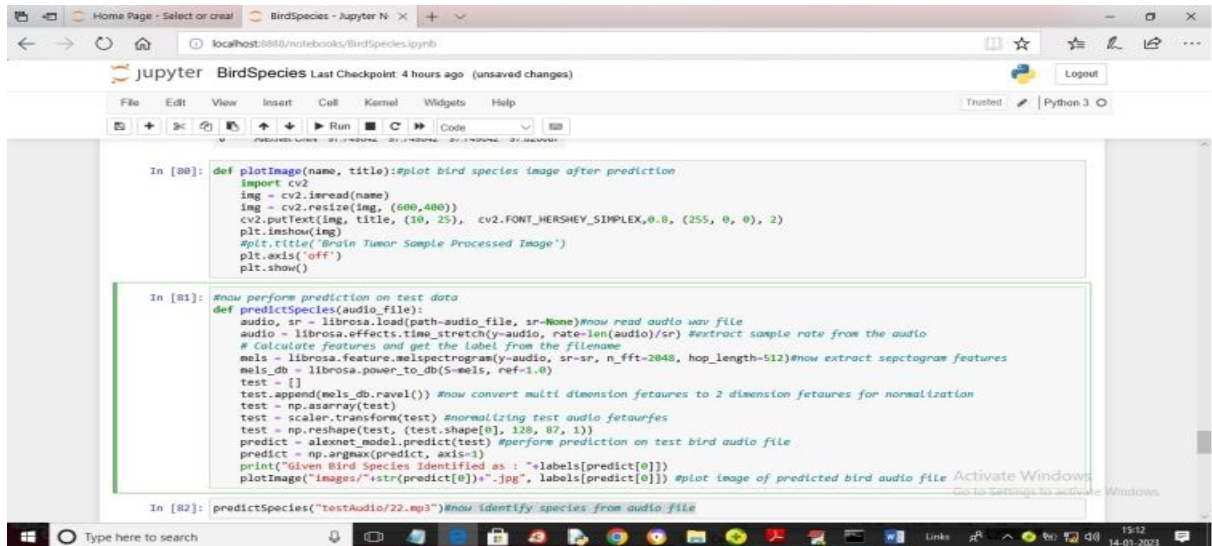
In above screen we are performing prediction on test data and then we got prediction accuracy as 97.82% and in confusion matrix graph x-axis represents Predicted Species Labels and y-axis represents True Labels and all blue colour boxes contains INCORRECT prediction count and different colour boxes contains CORRECT prediction count and we can see incorrect prediction are very so model able to get more than 97% accuracy





**Fig 8.12: performance metrics**

In above screen we are displaying accuracy, precision, recall and FSCORE of AlexNet model in tabular format



The screenshot shows a Jupyter Notebook interface with the following code cells:

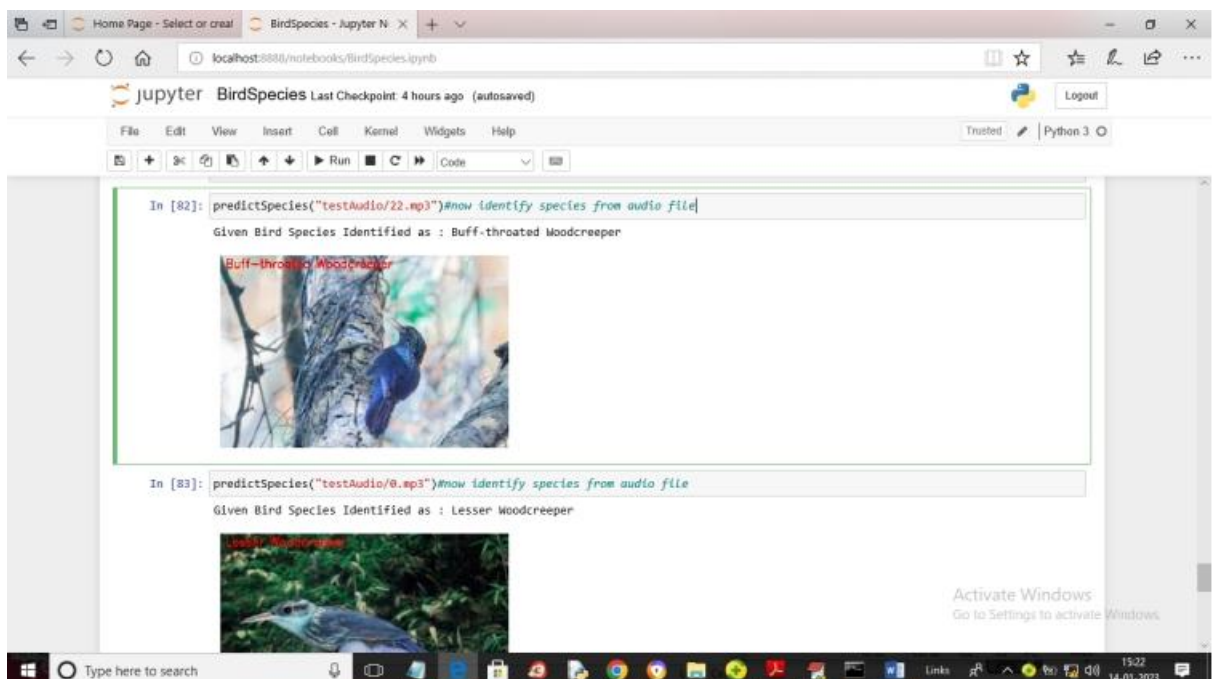
```
In [80]: def plotImage(name, title):#plot bird species image after prediction
import cv2
img = cv2.imread(name)
img = cv2.resize(img, (600,400))
cv2.putText(img, title, (10, 25), cv2.FONT_HERSHEY_SIMPLEX,0.8, (255, 0, 0), 2)
plt.imshow(img)
plt.title('Brain Tumor Sample Processed Image')
plt.axis('off')
plt.show()

In [81]: #now perform prediction on test data
def predictSpecies(audio_file):
    audio, sr = librosa.load(path=audio_file, sr=None)#now read audio wav file
    audio = librosa.effects.time_stretch(y=audio, rate=len(audio)/sr) #extract sample rate from the audio
    # Calculate features and get the label from the filename
    mels_db = librosa.feature.melspectrogram(y=audio, sr=sr, n_fft=2048, hop_length=512)#now extract spectrogram features
    mels_db = librosa.power_to_db(mels_db, ref=1.0)
    test = []
    test.append(mels_db.ravel()) #now convert multi dimension fetatures to 2 dimension fetatures for normalization
    test = np.asarray(test)
    test = scaler.transform(test) #normalizing test audio fetatures
    test = np.reshape(test, (test.shape[0], 128, 87, 1))
    predict = alexnet_model.predict(test) #perform prediction on test bird audio file
    predict = np.argmax(predict, axis=3)
    print("Given Bird Species Identified as : "+labels[predict[0]])
    plotImage("images/"+str(predict[0])+".jpg", labels[predict[0]]) #plot image of predicted bird audio file

In [82]: predictSpecies("testAudio/22.mp3")#now identify species from audio file
```

**Fig 8.13: prediction function**

In above screen we are defining function to perform prediction on given test audio file



The screenshot shows the execution of the prediction function from the previous figure. It displays the output for two audio files:

```
In [82]: predictSpecies("testAudio/22.mp3")#now identify species from audio file
Given Bird Species Identified as : Buff-throated Woodcreeper
```

Below the text output is an image of a Buff-throated Woodcreeper.

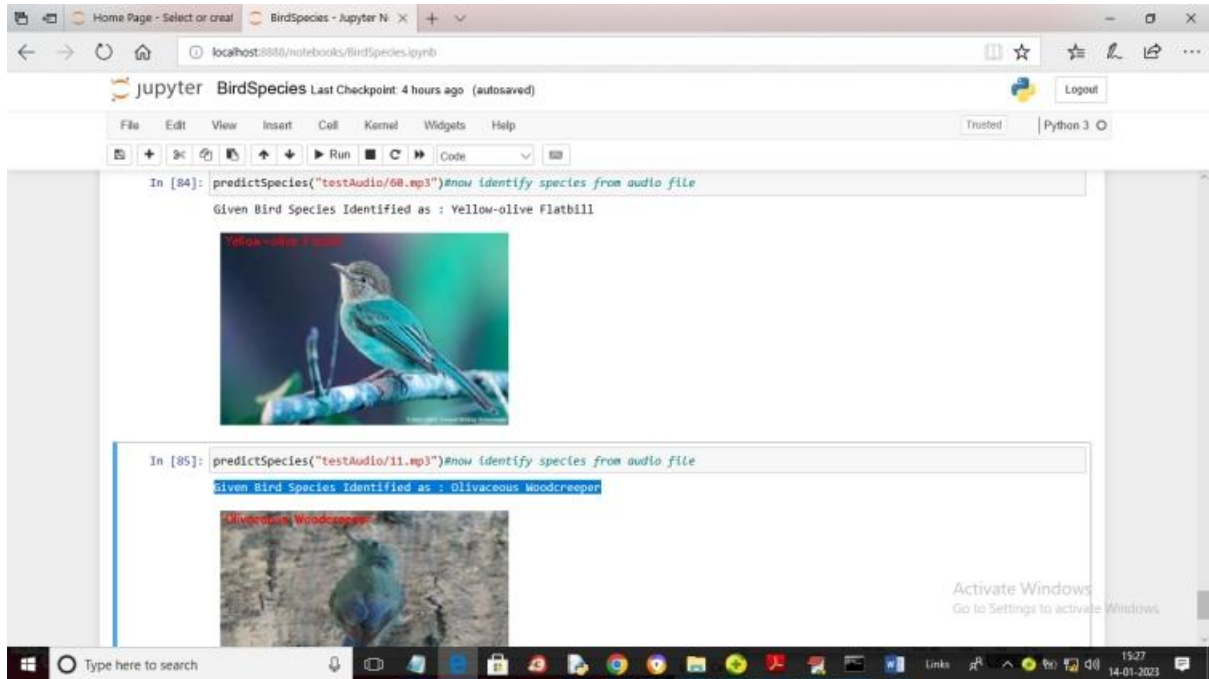
```
In [83]: predictSpecies("testAudio/0.mp3")#now identify species from audio file
Given Bird Species Identified as : Lesser Woodcreeper
```

Below the text output is an image of a Lesser Woodcreeper.

**Fig 8.14: testing**

In above screen we are calling 'PredictSpecies' function by giving bird audio file as input and then application will predict species and then plot predicted bird species images and in red colour we can see name of predicted species and in above image we can text output also.

Similarly you can input other audio files and predict species and below are other prediction



## 9.TESTING

System testing, also referred to as system-level tests or system-integration testing, is the process in which a quality assurance (QA) team evaluates how the various components of an application interact together in the full, integrated system or application. System testing verifies that an application performs tasks as designed. This step, a kind of black box testing, focuses on the functionality of an application. System testing, for example, might check that every kind of user input produces the intended output across the application.

### 9.1 PHASES OF SYSTEM TESTING

A video tutorial about this test level. System testing examines every component of an application to make sure that they work as a complete and unified whole. A QA team typically conducts system testing after it checks individual modules with functional or user-story testing and then each component through integration testing.

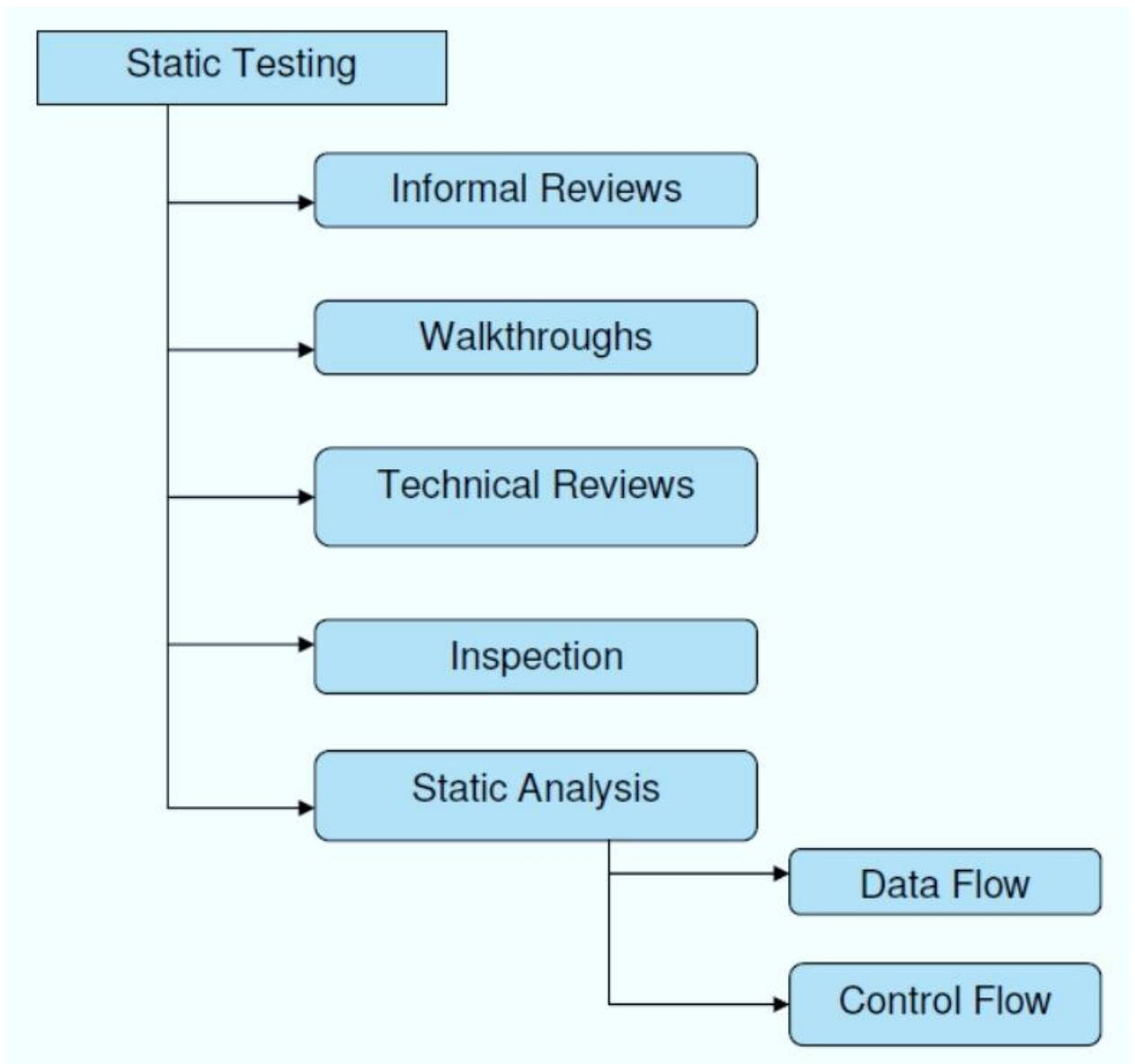
If a software build achieves the desired results in system testing, it gets a final check via acceptance testing before it goes to production, where users consume the software. An app-dev team logs all defects, and establishes what kinds and amount of defects are tolerable.

#### ***Software Testing Strategies:***

Optimization of the approach to testing in software engineering is the best way to make it effective. A software testing strategy defines what, when, and how to do whatever is necessary to make an end-product of high quality. Usually, the following software testing strategies and their combinations are used to achieve this major objective:

#### ***Static Testing:***

The early-stage testing strategy is static testing: it is performed without actually running the developing product. Basically, such desk-checking is required to detect bugs and issues that are present in the code itself. Such a check-up is important at the pre-deployment stage as it helps avoid problems caused by errors in the code and software structure deficits.

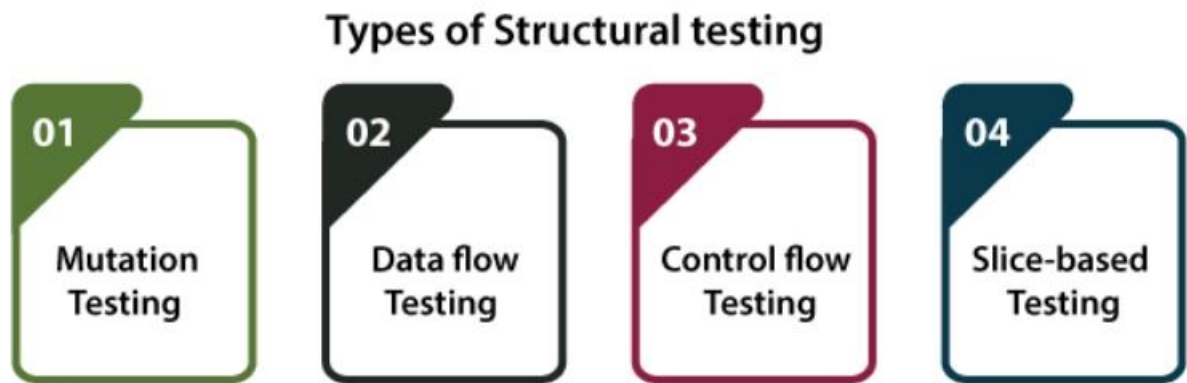


***Fig 9.1.1: static testing***

***Structural Testing:***

It is not possible to effectively test software without running it. Structural testing, also known as white-box testing, is required to detect and fix bugs and errors emerging during the pre-production stage of the software development process. At this stage, unit testing based on the software structure is performed using regression testing. In most cases, it is an automated process working within the test automation framework to speed up the development process at this stage. Developers and QA engineers have full access to the software's structure and data flows (data flows testing), so they could track any changes (mutation testing) in the system's behavior by comparing the tests' outcomes with the results of previous iterations (control flow testing).

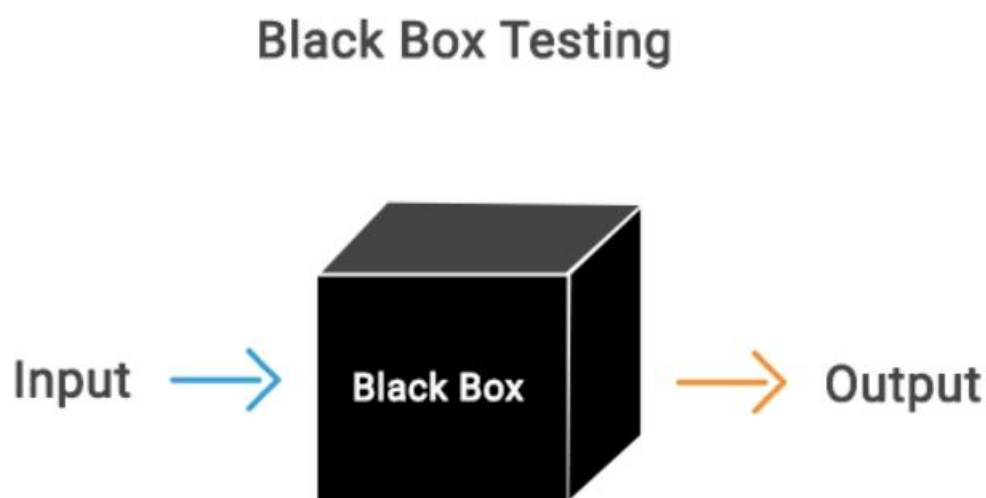




*Fig 9.1.2: structural testing*

#### ***Behavioral Testing:***

The final stage of testing focuses on the software's reactions to various activities rather than on the mechanisms behind these reactions. In other words, behavioral testing, also known as black-box testing, presupposes running numerous tests, mostly manual, to see the product from the user's point of view. QA engineers usually have some specific information about a business or other purposes of the software ('the black box') to run usability tests, for example, and react to bugs as regular users of the product will do. Behavioral testing also may include automation (regression tests) to eliminate human error if repetitive activities are required. For example, you may need to fill 100 registration forms on the website to see how the product copes with such an activity, so the automation of this test is preferable.



*Fig 9.1.3: blacbox testing*

## 9.2 TEST CASES:

| S.NO | INPUT                      | If available                                 | If not available    |
|------|----------------------------|----------------------------------------------|---------------------|
| 1    | Upload dataset             | Dataset loaded                               | There is no process |
| 2    | Model generation           | Alexnet with CNN model loaded for prediction | There is no process |
| 3    | Enter input for prediction | Prediction result displayed for test data    | There is no process |

### 9.2.1: Test cases

## 10.CONCLUSION

In this project, four different bird species were identified. The approach involved pre-processing of the bird sounds followed by the spectrogram generation of the same and these were used to train the model for classification. The data used for training consisted of real bird sounds recorded in their natural habitat amidst all other sounds. The outputs were observed for different values of learning rates, number of epochs and data split. The system was able to classify bird species based on the spectrogram image generated from their sounds with an accuracy of 97.82%. This accuracy was obtained considering human voices along with bird sounds. The accuracy can be further enhanced by fine tuning the performance parameters.

## 11. LIMITATIONS AND FUTURE SCOPE

- In the future, a mobile app could be created and released that would let consumers utilize their mobiles as handheld devices for bird sound analysis and prediction.
- The model would be trained on larger datasets to support wide range of species using big data techniques.
- We could use the advanced models that will be released in future which can efficiently handle large datasets and make the training process easier.

## 12. REFERENCES

- [1] [www.iucn.org/theme/species/our-work/birds](http://www.iucn.org/theme/species/our-work/birds)
- [2] [www.xeno-canto.org/](http://www.xeno-canto.org/)
- [3] Sujoy Debnath, Partha Protim Roy , Amin Ahsan Ali, M Ashraful Amin” Identification of Bird Species from Their Singing”, 5th International Conference on Informatics, Electronics and Vision (ICIEV), 2016
- [4] Rong Sun, Yihenew Wondie Marye, Hua-An-Zhao “FFT Based Automatic Species Identification Improvement with 4- layer Neural Network”, 2013 13th International Symposium on Communications and Information Technologies (ISCIT)
- [5] [www.mathworks.com/help/deeplearning/ref/alexnet.html](http://www.mathworks.com/help/deeplearning/ref/alexnet.html)
- [6] F. Briggs, B. Lakshminarayanan, L. Neal, X. Z. Fern, R. Raich, S. J. Hadley, A. S. Hadley, and M. G. Betts, “Acoustic classification of multiple simultaneous bird species: A multi-instance multi-label approach,” *The Journal of the Acoustical Society of America*, vol. 131, no. 6, pp. 4640–4650, 2012.
- [7] M. T. Lopes, A. Lameiras Koerich, C. Nascimento Silla, and C. Alves Kaestner, “Feature set comparison for automatic bird species identification,” in *Systems, Man, and Cybernetics (SMC)*, 2011 IEEE International Conference on. IEEE, 2011, pp. 965–970.
- [8] R. Bardeli, D. Wolff, F. Kurth, M. Koch, K.-H. Tauchert, and K.-H. Frommolt, “Detecting bird sounds in a complex acoustic environment and application to bioacoustic monitoring,” *Pattern Recognition Letters*, vol. 31, no. 12, pp. 1524–1534, 2010.
- [9] T. Scott Brandes, “Automated sound recording and analysis techniques for bird surveys and conservation,” *Bird Conservation International*, vol. 18, no. S1, pp. S163–S173, 2008.
- [10] A. Harma, “Automatic identification of bird species based on sinusoidal modeling of syllables,” in *Acoustics, Speech, and Signal Processing*, 2003. *Proceedings.(ICASSP’03)*. 2003 IEEE International Conference on, vol. 5. IEEE, 2003, pp. V–545.
- [11] A. Selin, J. Turunen, and J. T. Tantt, “Wavelets in recognition of bird sounds,” *EURASIP Journal on Applied Signal Processing*, vol. 2007, no. 1, pp. 141–141, 2007.

- [12] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan, “Object detection with discriminatively trained part-based models,” *Pattern Analysis and Machine Intelligence, IEEE Transactions on*, vol. 32, no. 9, pp. 1627–1645, 2010.
- [13] R. Brunelli, *Template matching techniques in computer vision*. Wiley Online Library, 2009.
- [14] J. Lewis, “Fast normalized cross-correlation,” in *Vision interface*, vol. 10, no. 1, 1995, pp. 120–123.
- [15] L. Breiman, “Random forests,” *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

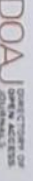
## APPENDIX

### Conference presentation certificate

*Certificate of Presentation*  
International Conference on  
Recent Advances in Science, Technology, Engineering and Management  
ICRASTEM-2K23 on 11<sup>th</sup> April 2023

Organised By  
Department of Computer Science and Engineering  
**VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY**  
(AUTONOMOUS)  
Permanently Affiliated to JNTU Kakinada, Approved by AICTE, Accredited by NAAC with 'A' Grade, ISO 9001:2015 Certified.  
All eligible B Tech branches Accredited by NBA, Nambur, Pedakani (M), Guntur (DL) - 522508.

 **VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY**

This is to certify that the paper entitled Bird Species Identification using Audio Processing and AlexNet Accural Attention with author(s) Sudhakar Reddy, S. S. Vinay Reddy, T. Chandrika, Karthikeya, S. S. Sathya Akhila was presented by ..... in the ICRATEST - 2K23 International Conference on Recent Advances in Science, Technology, Engineering and Management, held on 11<sup>th</sup> April 2023 at Vasireddy Venkatadri Institute of Technology, Guntur Dt. In association with SOLETE (Society for Learning Technologies) Vijayawada.

  
Mr. T. Kranthi Kumar  
CEO, SOLETE

  
Dr. V. Ramachandran  
Convener  
ICRASTEM-2K23  
Professor & HOD CSE, VIT

  
Dr. Y. Mallikarjuna Reddy  
Principal, VIT



## Bird Species Identification using Audio Processing and AlexNet Neural Network

**Sanjay Gandhi Gundabatini<sup>1</sup>**, Professor, Department of CSE,  
Vasireddy Venkatadri Institute of Technology, Nambur, Guntur Dt., Andhra Pradesh.  
**Sangam Sai Sri Vinay Reddy<sup>2</sup>, Thota Chandrika<sup>3</sup>, Somarouthu Kaarthikeya Pavana  
Kumaar<sup>4</sup>, Shaik Siddik<sup>5</sup>, Torlikonda Satya Akhil<sup>6</sup>**

<sup>2,3,4,5,6</sup> UG Students, Department of CSE,  
Vasireddy Venkatadri Institute of Technology, Nambur, Guntur Dt., Andhra Pradesh.

<sup>1</sup> sanjaygandhi.g@vvit.net, <sup>2</sup> sssvinayr@gmail.com,  
<sup>3</sup> chandrikathota1005@gmail.com, <sup>4</sup> kaarthikeyapavanakumaar@gmail.com, <sup>5</sup> shaiksiddik  
630@gmail.com, <sup>6</sup> satyaakhil1602@gmail.com

DOI: 10.48047/IJFANS/V11/I12/173

### Abstract

This research involved identifying the birds with the help of audio recorded from the real world environment. Most of the methods use images for detection of birds. But, some species may be similar to see. Hence, we took audio as the basis for classification. The audio frequency was plotted as spectrogram and it was inspected to extract the patterns and classify the bird. Legacy practices involved manual inspection of spectrogram that are plotted on the frequency of audio signals. But, this is time taking process and often produces inaccurate results. Hence, we created a computerized process to inspect the spectrogram. The computer learns from patterns of spectrogram during the training process and learns to detect a new and unseen audio. This entire procedure involved two crucial phases. The first stage of the process was to create a dataset with audio files collected from websites like Xeno-canto org that includes all sound recordings of birds. In this research work, we considered 4 species of wood pecker in the Germany region. Hence, we have collected approximately 120 recordings for each species, thus a total of 500 recordings were collected. The collected sounds undergone a series of pre-processing phases like reconstruction, framing, and silence removal, pre-emphasis for removing any noises like human actions, wind sounds, tree sounds. For every processed sound clip, the spectrogram is plotted and it was given as input to a neural network that is in the second stage, which in turn detects the recording at the end. Since the image was given as input, we used Convolutional Neural Network (CNN) which is a best neural net in deep learning for text and image based tasks. The CNN categorizes sound clip and determines the species of bird based on input features. A model was created and put into practice.

**Keywords:** Convolution, Neural Network, Spectrogram, Audio processing, Noise removal, Deep Learning, Classification.



## Introduction

IUCN estimates that there are close to 10,000 bird varieties of that can be found in a vast range of environments, from Brazilian jungle to the cold beaches of Antarctica [1]. These species are vital to an ecosystem's well operation and exhibit incredible diversity in terms of behaviour and form. But recent human activities that range from encroaching on their ecosystems to completely destroying those habitats have jeopardised this wonderful biological diversity, and when combined with natural calamities and climate change, many bird species have been undergone extinction. An estimated 13% of all bird species, or close to 1,370 species, are thought to be in danger of going extinct. Despite the fact that many different bird species are frequently observed, people have a hard time identifying them. The objective of this work is to create computerised methods for classifying different species of birds by relying on their noises.



Fig.1: process flow figure

The rapid changes in bird population has been a major concern in recent times by many environmental organizations. However, acquiring information about different types of birds is a labor-intensive and expensive process that demands a lot of human effort. In this scenario, we need a good tool for preparing data is required. In this aspect, determining which bird species a particular image belong to, plays a vital role in determining which categories it belongs to. [14-22]

Bird species identification is determining which category a particular species of bird falls under using a photograph. Images, audio, and videos can all be used to identify different bird species. It is possible to identify birds by listening for their distinctive sounds thanks to an audio processing technique. However, handling of such data becomes increasingly complicated because of the mixed sounds in the condition, such as creepy crawlies, real-world items, and so forth. Images typically aid in information discovery more than sounds or recordings. In this perspective, utilizing an image rather than voice or video to categorize birds is preferred. Bird species identification is a difficult issue for both humans and

computational processes that perform such a task automatically. But, for more accuracy, despite of the above stated complications, we proceeded to audio based classification.

### **Literature Survey**

#### **A. Identification of Bird Species from Their Singing:**

One of the enigmatic phenomena that we are learning more about as time goes on is biodiversity. The BIOTOPE organization has launched an initiative to aid in the process of comprehending nature. The group's open source platform will help to gather information by identifying different types of birds from their humming. The fundamental concept of this work is to recognize different types of birds by their songs or calls. The research discussed in this article investigates an autonomous system for classifying various bird species based on their songs and calls. This will make it easier to catalogue and identify new species or subspecies. Additionally, it will support research on bird behaviour and ecology and help with population surveillance. In this work, we put into practise a system that can find different bird species from audio recordings. According on the ROC performance analysis of the proposed system, Random Decision Tree may produce 0.96 area under the curve.

#### **B. FFT Based Automatic Species Identification Improvement with 4- layer Neural Network:**

An automated approach for identifying species has been devised in this paper. Recorded data was automatically split, analyzed, had features removed, and recognized. A 4-layer neural network-based feature quantity technique based on FFT and frequency band power derivative is provided. Wild bird species identification using a 4-layer neural network based on reliable data has been tested, and the results have shown promise.

#### **C. Feature set comparison for automatic bird species identification:**

In order to identify the species of a bird from its recorded audio song, an issue known as automated bird species identification must be solved. Given that it is an indirect, non-invasive method of evaluation, this is an ingenious way to keep track of ecosystem biodiversity. This study compares various feature sets that describe the audio characteristics of the audio stream in various ways with machine learning methods like neural networks and SVM. Three different bird species' recorded songs are used as the subject of experiments. The experimental results demonstrate that it is possible to achieve highly encouraging results in automated bird species identification. The results compare the performance of the feature sets and several classifiers.

#### **D. Detecting bird sounds in a complex acoustic environment and application to bio-acoustic monitoring:**

Bird population trends are a crucial indication for environmental protection, but determining these trends requires a lot of time and human effort. Automated approaches for the detection of bird vocalizations are now conceivable through enormous advancements

in audio signal processing and pattern recognition. These tactics can be used to support population censuses. A study that examined the viability of automatic bird song detection in conjunction with bird monitoring was highlighted in this task. Describing new methods for the automatic habitat mapping of the vocalizations of two bird species that are in danger of extinction was given high importance. These techniques work by identifying temporal trends in a certain frequency band that are specific characteristics of the species. The noise that is present in real-world audio scenarios is specifically suppressed. Insights demonstrate that high recognition rates with a manageable number of false positive detection are still feasible under real-world recording situations.

#### **E. Automated sound recording and analysis techniques for bird surveys and conservation:**

Automated sound recording and analysis of bird sounds need to be used more and developed further. Since birds are essential to the health of ecosystems, science and conservation efforts would substantially benefit from methods that make avian monitoring more effective and precise. We give a briefing on popular methods used to identify and categorize bird sounds automatically, as well as a summarized view of the hardware approaches to automated sound recording. An interface for a scheduling timer to run a standalone recorder, a recording device that is programmable, and a computer with single board are three examples of the three basic kinds of hardware solutions we compare and contrast for automating sound recording. We also give examples of the two basic methods using wave guides and tiny arrays of microphone elements for enhancing microphone performance for automated recorders.

#### **F. Deep Learning Based Audio Classifier for Bird Species**

##### **Problem Identification**

Automatically identifying bird cries from continuous recordings gathered from the environment would be a huge advancement in the study methodologies. As noise or clipping are frequently present in these recordings, reliable automated processes must be used instead of manual, conventional techniques. Automated systems are required because manual spectrogram examinations are frequently mistaken and typically require a large number of professionals to employ specialized techniques.

##### **Disadvantages:**

1. Since manual checks of the spectrograms are frequently prone to error and typically involve many experts and esoteric methodologies, automated systems are necessary.

### Methodology

In this paper, a method for automating the entire process of identifying bird sounds using sound processing and convolutional neural networks is proposed. The initial step entails compiling all of the sound recordings into a database. Then, sound pre-processing methods like pre-emphasis, framing, silence removal, and reconstruction are applied to these recordings.

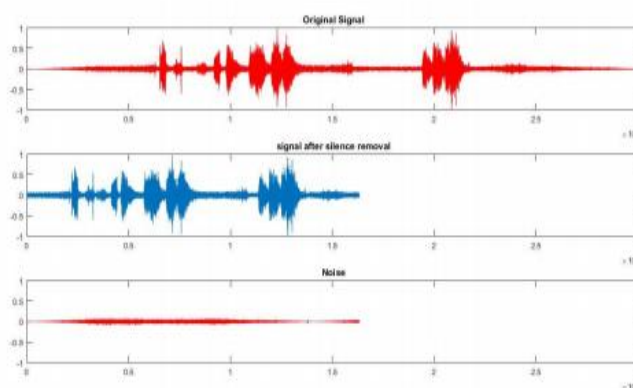


Fig 2: silence removal

### Pre-emphasis

It is a technique to process signals that involves improvising the frequency components of an audio signal. It amplifies high audio components and diminishes the intensity of low frequency components.

### Framing

This process divides audio signal into small and manageable parts. It assists in processing the signals with ease. But, very short duration of frames results in higher computational burden.

### Silence removal

It is an approach to detect and remove the periods of silence from an audio signal. One common approach followed is to set a threshold frequency. If any signal is detected below this frequency, it is removed from the sample. This threshold is set based on the averages of all frequencies in the signal.

### Reconstruction

This process involves digitally encoded audio signal back into its original wave form. It generates continuous wave form from the digital signal which matches the nearest frequency.

For each sound clip, spectrograms are produced, and these spectrograms are fed into a CNN that has been trained on a GPU. For the trained CNN model, a real-time implementation model was developed.

#### Advantages:

1. This system has a 98% accuracy rate when classifying bird species based on the spectrogram image created from their sounds.
2. This precision was attained by taking into account both human voices and avian sounds.
3. By fine-tuning the performance settings, the precision can be increased even more.

#### Process flow

- Data exploration: this module loads data into the system.
- Processing: this module will read and process the data.
- Train\_test data split: to divide the dataset into train and test data
- Generating the Model: Building the model Alexnet with CNN.

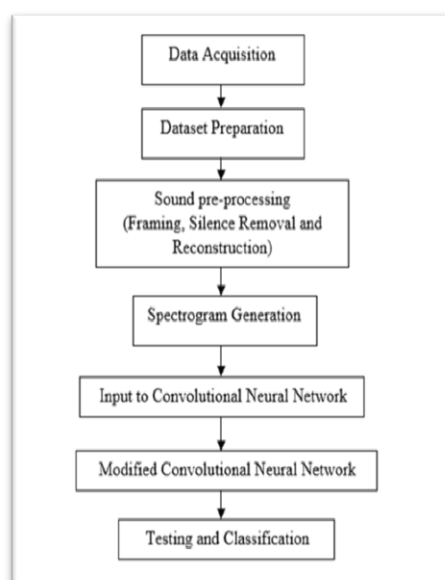


Fig.3: System architecture

- Predict accuracy: Using this module we will get accuracy, precision, recall and FSCORE
- User input: To take unseen data as input to test the model.

**Implementation**

To identify or predict the species of bird, we are using the AlexNet CNN model, and in order to do so, we are using two different modules. The first module is used to read bird audio files, after which audio framing and noise removal are applied, and finally spectrogram features are extracted from the processed audio file. Applying the pre-trained CNN model from AlexNet on the extracted features in the second module to create a bird species recognition model. using the datasets of XENO CANTO bird audio files, which incorporates noise signals, to train the model.

**ALEXNET:**

A special type of neural network called a convolutional neural network (CNN) is used for supervised learning. CNNs are useful for processing images, natural language, and other cognitive tasks. AlexNet is the first neuralnet to make use of GPU training. 5 convolutional layers (3 are max-pooling layers, 2 are normalization layers), and we added 3 extra layers with (9,9), (7,7) and (6,6) number of neurons respectively, 2 layers that are fully connected, and a Softmax layer forms required architecture.

A multi-layer perceptron with a specific architecture for identifying two-dimensional picture data is the convolution neural network algorithm. An input layer, a convolution layer, a sample layer, and an output layer are among its four layers.

The sample layer and convolution layer in a deep network architecture could have more than one. Convolution neural network techniques do not have the same limitations as the Boltzmann machine in that each neuron just needs to sense the local portion of the image rather than the entire image, as is the case with the Boltzmann machine.

Additionally, each neuron's parameters are specified to be the same, including the sharing of weights and the use of identical convolution kernels for the deconvolution picture for each neuron.

The central elements of CNN are the local receptive field, weight sharing, and time- or space-based sub sampling with the aim of feature extraction and training parameter reduction.

The benefit of the CNN algorithm is that it learns intuitively from training data rather than explicitly extracting features. The network can learn in parallel and become less complex because the same neuron weights are on the surface of the feature mapping. utilizing a sub-sampling framework based on deformation displacement, scale, and time resilience.

Network topology and input data may go well together. It offers special benefits for picture processing. These actions are part of the Convolution Neural Network:

#### *Convolution Layer:*

It is the basic building block of CNN. It performs mathematical operation known as convolution which handles multi dimensional data like images, sound waves. This operation is useful to extract features from the given data. This layer consists of filters which are responsible for identifying patterns from the data. No. of filters in the layer is a hyper parameter which can be tuned for better control, performance.

In addition to convolution operation, it includes other operations like batch normalization and activation functions which will aid in improvising the stability of the network.

#### *Pooling Layer:*

Pooling layer is responsible for compressing the input data while holding the necessary features in it. This helps in reducing computational efforts and over-fitting of the network. Various types of pooling are:

- Max Pooling
- Mean Pooling
- Sum Pooling

#### *Fully Connected Layer:*

It is a layer in CNN whose individual node is connected to every other node in another layer. They are last layers in the network. . No. of neurons in the layer is a hyper parameter which can be tuned for better control, performance.

## Results

```
In [1]: # def getID(name): refction to get ID of the bird as label from species names
        index = 0
        for i in range(len(labels)):
            if labels[i] == name:
                index = i
                break
        return index

labels = []
for root, dirs, directory in os.walk("dataset\newkalis"):
    #now loop all files and get labels and then display all birds names
    for i in range(len(directory)):
        name = os.path.basename(root)
        if name not in labels:
            labels.append(name)
print('Bird species found in dataset')
print()
for i in range(len(labels)):
    print(labels[i])

Bird Species found in dataset:

buff-throated woodcreeper
lesser woodcreeper
olivaceous woodcreeper
yellow-olive flatbill
```

Fig.4: Dataset



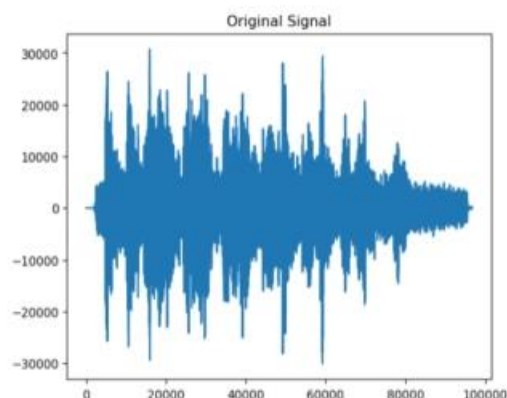


Fig.5: Audio file with noise

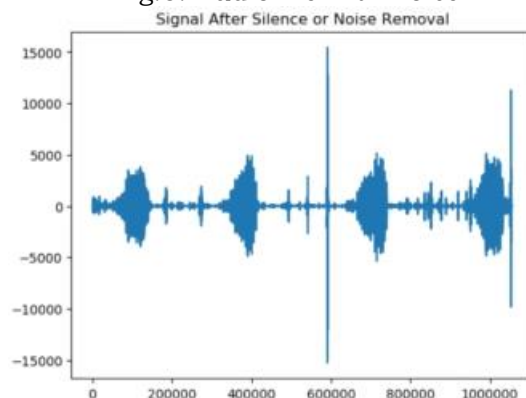


Fig.6: Audio without noise

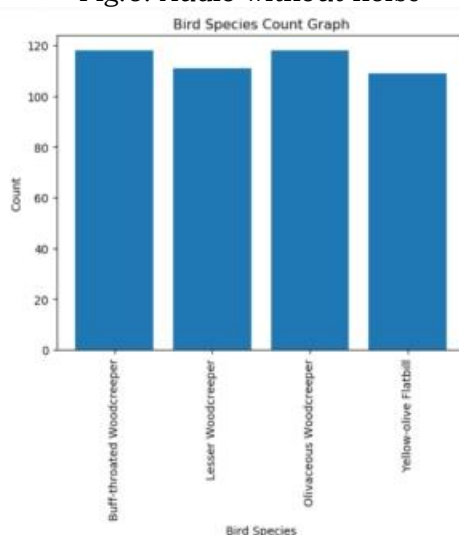


Fig.7: graph of various birds species and their count in the dataset



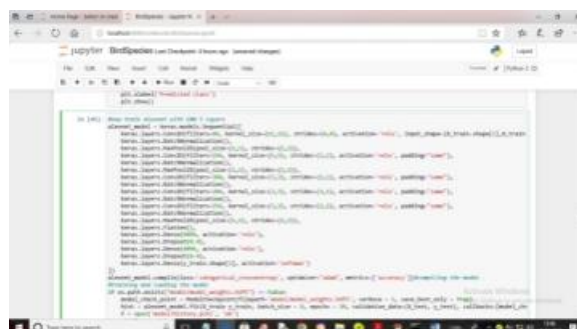


Fig.8: Model trained

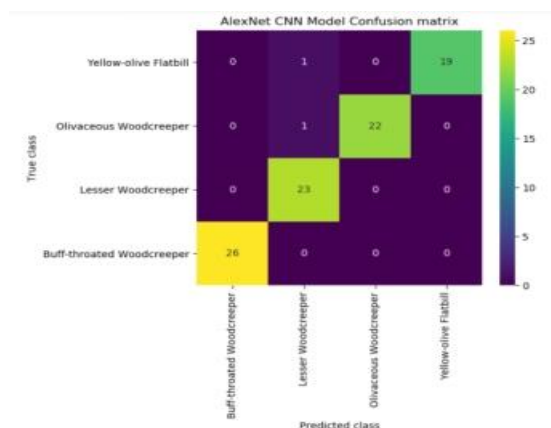


Fig.9: Confusion matrix



Fig.10: Prediction for test data

## Conclusion

The system has a 98% accuracy rate when classifying bird species based on the spectrogram image created from their sounds. This precision was attained by taking into account both human voices and avian sounds. By fine-tuning the performance settings, the precision can be increased even more.

### Limitations and Future Scope

- In the future, a mobile app could be created and released that would let consumers utilize their mobiles as handheld devices for bird sound analysis and prediction.
- The model would be trained on larger datasets to support wide range of species using big data techniques.
- We could use the advanced models that will be released in future which can efficiently handle large datasets and make the training process easier.

### References

- [1] Sujoy Debnath, Partha Protim Roy , Amin Ahsan Ali, M Ashraful Amin” Identification of Bird Species from Their Singing”, 5th International Conference on Informatics, Electronics and Vision (ICIEV), 2016
- [2] Rong Sun, Yihenew Wondie Marye, Hua-An-Zhao “FFT Based Automatic Species Identification Improvement with 4- layer Neural Network”, 2013 13th International Symposium on Communications and Information Technologies (ISCIT)
- [3] F. Briggs, B. Lakshminarayanan, L. Neal, X. Z. Fern, R. Raich, S. J. Hadley, A. S. Hadley, and M. G. Betts, “Acoustic classification of multiple simultaneous bird species: A multi-instance multi-label approach,” The Journal of the Acoustical Society of America, vol. 131, no. 6, pp. 4640–4650, 2012.
- [4] M. T. Lopes, A. Lameiras Koerich, C. Nascimento Silla, and C. Alves Kaestner, “Feature set comparison for automatic bird species identification,” in Systems, Man, and Cybernetics (SMC), 2011 IEEE International Conference on. IEEE, 2011, pp. 965–970.
- [5] R. Bardeli, D. Wolff, F. Kurth, M. Koch, K.-H. Tauchert, and K.-H. Frommolt, “Detecting bird sounds in a complex acoustic environment and application to bioacoustic monitoring,” Pattern Recognition Letters, vol. 31, no. 12, pp. 1524–1534, 2010.
- [6] T. Scott Brandes, “Automated sound recording and analysis techniques for bird surveys and conservation,” Bird Conservation International, vol. 18, no. S1, pp. S163–S173, 2008.
- [7] A. Harma, “Automatic identification of bird species based on sinusoidal modeling of syllables,” in Acoustics, Speech, and Signal Processing, 2003. Proceedings.(ICASSP’03). 2003 IEEE International Conference on, vol. 5. IEEE, 2003, pp. V–545.
- [8] A. Selin, J. Turunen, and J. T. Tantt, “Wavelets in recognition of bird sounds,” EURASIP Journal on Applied Signal Processing, vol. 2007, no. 1, pp. 141–141, 2007.

- [9] P. F. Felzenszwalb, R. B. Girshick, D. McAllester, and D. Ramanan, "Object detection with discriminatively trained part-based models," Pattern Analysis and Machine Intelligence, IEEE Transactions on, vol. 32, no. 9, pp. 1627–1645, 2010.
- [10] R. Brunelli, Template matching techniques in computer vision. Wiley Online Library, 2009.
- [12] J. Lewis, "Fast normalized cross-correlation," in Vision interface, vol. 10, no. 1, 1995, pp. 120–123.
- [13] L. Breiman, "Random forests," Machine learning, vol. 45, no. 1, pp. 5–32, 2001.
- [14] Sri Hari Nallamala, et al., "A Literature Survey on Data Mining Approach to Effectively Handle Cancer Treatment", (IJET) (UAE), ISSN: 2227 – 524X, Vol. 7, No 2.7, SI 7, Page No: 729 – 732, March 2018.
- [15] Sri Hari Nallamala, et.al., "An Appraisal on Recurrent Pattern Analysis Algorithm from the Net Monitor Records", (IJET) (UAE), ISSN: 2227 – 524X, Vol. 7, No 2.7, SI 7, Page No: 542 – 545, March 2018.
- [16] Sri Hari Nallamala, et.al, "Qualitative Metrics on Breast Cancer Diagnosis with Neuro Fuzzy Inference Systems", International Journal of Advanced Trends in Computer Science and Engineering, (IJATCSE), ISSN (ONLINE): 2278 – 3091, Vol. 8 No. 2, Page No: 259 – 264, March / April 2019.
- [17] Sri Hari Nallamala, et.al, "Breast Cancer Detection using Machine Learning Way", International Journal of Recent Technology and Engineering (IJRTE), ISSN: 2277-3878, Volume-8, Issue-2S3, Page No: 1402 – 1405, July 2019.
- [18] Sri Hari Nallamala, et.al, "Pedagogy and Reduction of K-nn Algorithm for Filtering Samples in the Breast Cancer Treatment", International Journal of Scientific and Technology Research, (IJSTR), ISSN: 2277-8616, Vol. 8, Issue 11, Page No: 2168 – 2173, November 2019.
- [19] Kolla Bhanu Prakash, Sri Hari Nallamala, et al., "Accurate Hand Gesture Recognition using CNN and RNN Approaches" International Journal of Advanced Trends in Computer Science and Engineering, 9(3), May – June 2020, 3216 – 3222.
- [20] Sri Hari Nallamala, et al., "A Review on 'Applications, Early Successes & Challenges of Big Data in Modern Healthcare Management'", Vol.83, May - June 2020 ISSN: 0193-4120 Page No. 11117 – 11121.
- [21] Nallamala, S.H., et al., "A Brief Analysis of Collaborative and Content Based Filtering Algorithms used in Recommender Systems", IOP Conference Series: Materials Science and Engineering, 2020, 981(2), 022008.
- [22] Nallamala, S.H., Mishra, P., Koneru, S.V., "Breast cancer detection using machine learning approaches", International Journal of Recent Technology and Engineering, 2019, 7(5), pp. 478–481.